

PeepI — All 80 Cursor Composer Prompts

Repo: `github.com/jhouston2019/peepI2025.v1`

Stack: React Native / TypeScript (PeepIMobile), Node.js / Express / Firebase (backend)

Rule: One task at a time. Complete it, test it, commit it.

Commit format: `[Phase X / Task Y] Description`

PHASE 1 — Critical Bug Fixes

App must work end-to-end before anything else. Complete all 14 tasks and validate with a full sign-up → peep flow before moving to Phase 2.

Task 1 — Fix AuthService export & call signature

CONTEXT

PeepI is a real-time crowd intelligence mobile app built with React Native / TypeScript (frontend) and Node.js / Express / Firebase (backend). This is Phase 1, Task 1 — the first critical bug fix needed before the app can function.

PROBLEM

`AuthService` class is never exported from `AuthService.ts` — only the singleton instance `authService` is exported. Meanwhile, `App.tsx` imports `{ AuthService }` (the class, not the instance) and calls `AuthService.login(email, password)` passing two separate string arguments. The actual `login()` method signature expects a single `LoginCredentials` object `{ email, password }`. Both problems must be fixed simultaneously.

FILES TO EDIT

- `PeepIMobile/src/services/AuthService.ts`
- `PeepIMobile/App.tsx`

EXACT CHANGES REQUIRED

- In `AuthService.ts`, locate the class declaration line. Change:

```
ts
```

```
class AuthService {
```

to:

ts

```
export class AuthService {
```

2. In `App.tsx`, locate the import line that reads:

ts

```
import { AuthService } from './src/services/AuthService';
```

Change it to:

ts

```
import { authService } from './src/services/AuthService';
```

3. In `App.tsx`, find every call to `AuthService.login(email, password)` and replace each one with:

ts

```
authService.login({ email, password })
```

4. Scan `App.tsx` for any other calls made on `AuthService` (the class) and replace them with calls on `authService` (the singleton) using the correct argument shapes as defined in `AuthService.ts`.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any existing functionality unrelated to this task
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. TypeScript compiler reports zero errors related to `AuthService` in both files.
2. Calling `authService.login({ email: 'test@test.com', password: 'abc123' })` from `App.tsx` compiles without type errors.

Task 2 — Fix Firestore peep create security rule

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 1, Task 2. The Firestore security rules have a critical bug that prevents new peeps from being created.

PROBLEM

The peep create rule uses `resource.data.userId`, which references the existing document — but on a `create` operation, no existing document exists yet, so this always fails. It must be changed to `request.resource.data.userId` to reference the incoming write. Additionally, the rules file is missing rules for the following collections that the app requires: `likes`, `comments`, `peep_history`, `merchant_accounts`, `merchant_ads`, `points`, `groups`.

FILES TO EDIT

- `firestore.rules`

EXACT CHANGES REQUIRED

- Find the rule block for the `peeps` collection create operation. Change every instance of:

```
resource.data.userId
```

to:

```
request.resource.data.userId
```

within the `allow create` condition only. Do not change `resource.data` references in `allow update` or `allow delete` rules.

- Add the following collection rules after the existing peeps rules block (adjust indentation to match existing style):

```

match /likes/{likeId} {
  allow read: if request.auth != null;
  allow create, delete: if request.auth != null && request.auth.uid ==
resource.data.userId;
}
match /comments/{commentId} {
  allow read: if request.auth != null;
  allow create: if request.auth != null && request.auth.uid ==
request.resource.data.userId;
  allow delete: if request.auth != null && request.auth.uid ==
resource.data.userId;
}
match /peep_history/{historyId} {
  allow read: if request.auth != null;
  allow write: if false;
}
match /merchant_accounts/{merchantId} {
  allow read: if request.auth != null && request.auth.uid == merchantId;
  allow create: if request.auth != null && request.auth.uid == merchantId;
  allow update: if request.auth != null && request.auth.uid == merchantId;
}
match /merchant_ads/{adId} {
  allow read: if request.auth != null;
  allow write: if request.auth != null;
}
match /points/{userId} {
  allow read: if request.auth != null;
  allow write: if false;
}
match /groups/{groupId} {
  allow read: if request.auth != null;
  allow create: if request.auth != null;
  allow update: if request.auth != null;
}

```

CONSTRAINTS

- Do not modify any files not listed above
- Do not alter any existing rules blocks except to fix the `resource.data.userId` reference on the create rule
- Maintain existing indentation and formatting style
- After changes, the app must still compile

VERIFICATION

1. `firebase emulators:start` runs without rules parse errors.
 2. A test create operation on the `peeps` collection succeeds when `request.resource.data.userId` matches `request.auth.uid`.
-

Task 3 — Resolve dual authentication system

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 1, Task 3. The backend and mobile app are using two incompatible authentication systems.

PROBLEM

The backend (`server.js`) issues custom JWTs verified with `jwt.verify()`. Firestore security rules use `request.auth`, which only recognizes Firebase Auth tokens — not custom JWTs. These two systems are completely disconnected. The fix is to standardize on Firebase Auth tokens end-to-end: the backend must verify Firebase ID tokens using `admin.auth().verifyIdToken()`, and the mobile client must obtain tokens from `@react-native-firebase/auth` and send them as Bearer tokens on all API calls.

FILES TO EDIT

- `server.js`
- `firestore.rules`
- `PeepIMobile/src/services/AuthService.ts`

EXACT CHANGES REQUIRED

1. In `server.js`, find the authentication middleware that calls `jwt.verify()`. Replace the entire verification call with:

```
js

const decodedToken = await admin.auth().verifyIdToken(token);
req.user = decodedToken;
```

Remove any `jwt` import or `JWT_SECRET` usage that is no longer needed after this change.

2. In `server.js`, find the `/auth/register` and `/auth/login` routes. Remove any code that calls `jwt.sign()` to generate and return a custom JWT. These routes should no longer mint their own tokens — Firebase Auth handles token issuance.
3. In `AuthService.ts`, update the sign-in method to use `@react-native-firebase/auth` for authentication:

```
ts

import auth from '@react-native-firebase/auth';

async login(credentials: LoginCredentials): Promise<void> {
  const { email, password } = credentials;
  const userCredential = await auth().signInWithEmailAndPassword(email, password)
  const idToken = await userCredential.user.getIdToken();
  // Store idToken for use in API calls
}
```

4. In `AuthService.ts`, add a helper method `getIdToken()` that calls `auth().currentUser?.getIdToken()` and returns the current Firebase ID token. All API calls must use this token as a Bearer header.
5. In `firestore.rules`, verify (do not change unless broken) that all authenticated rules use `request.auth != null` — this will now work correctly since Firebase Auth tokens are being used.

CONSTRAINTS

- Do not modify any files not listed above
- Do not remove any routes from `server.js` — only change the authentication mechanism
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `server.js` contains no remaining calls to `jwt.verify()` or `jwt.sign()`.
 2. The auth middleware in `server.js` calls `admin.auth().verifyIdToken(token)` and attaches the decoded token to `req.user`.
-

Task 4 — Remove hashed passwords from Firestore

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 1, Task 4 — a security fix.

PROBLEM

In `server.js`, the `/auth/register` route writes a `{password: hashedPassword}` field directly to the user's Firestore document. Firebase Auth already manages credentials securely — storing any form of the password (even hashed) in Firestore is a security liability with no benefit. This line must be removed.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

1. In `server.js`, find the `/auth/register` route handler.
2. Locate the Firestore `set()` or `add()` call that writes the user document. Find the line that includes `{password: hashedPassword}` (or any variant such as `{password: hashed}`, `{passwordHash: hashedPassword}`, etc.) within that object.
3. Delete that line entirely from the object being written to Firestore.
4. If `bcrypt.hash()` or any other hashing library call exists solely to produce this value and is not used anywhere else in the register route, remove that call as well.
5. Do not remove any other fields from the user document write.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any other routes or middleware
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. The Firestore document written during registration contains no `password`, `passwordHash`, or `hashedPassword` field.
2. The `/auth/register` route still successfully creates a Firebase Auth user and the Firestore user document.

Task 5 — Fix profile update field injection

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 1, Task 5 — a security fix.

PROBLEM

The `PUT /users/profile` route in `server.js` spreads the entire `req.body` into the Firestore update: `{ ...req.body, updatedAt }`. This allows any caller to inject arbitrary fields (e.g., `isAdmin`, `points`, `stripeCustomerId`) into user documents. The fix is to explicitly whitelist only the fields that are safe to update via this endpoint.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

1. In `server.js`, find the `PUT /users/profile` route handler.
2. Replace the spread `{ ...req.body, updatedAt }` with an explicit whitelist object:

```
js

const allowedFields = ['username', 'firstName', 'lastName', 'bio', 'profileImageU
const updateData = {};
allowedFields.forEach(field => {
  if (req.body[field] !== undefined) {
    updateData[field] = req.body[field];
  }
});
updateData.updatedAt = admin.firestore.FieldValue.serverTimestamp();
```

3. Use `updateData` in the Firestore update call instead of `{ ...req.body, updatedAt }`.
4. Add a check before the update: if `Object.keys(updateData).length === 1` (only `updatedAt`), return a 400 response with `{ error: 'No valid fields provided' }`.

CONSTRAINTS

- Do not modify any files not listed above

- Do not change any other routes
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. Sending `{ isAdmin: true, username: 'test' }` to `PUT /users/profile` only saves `username` to Firestore — `isAdmin` is silently ignored.
 2. Sending `{ isAdmin: true }` alone returns HTTP 400.
-

Task 6 — Fix GET /venues/nearby validator bug and replace table scan

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 1, Task 6.

PROBLEM

The `GET /venues/nearby` route uses `body()` validators from `express-validator`, but `body()` reads from `req.body` — which is always empty on GET requests. The validators silently pass without reading anything, making `lat/lng` validation completely non-functional. Additionally, the route performs a full Firestore table scan to find nearby venues, which will not scale. Both issues must be fixed in this task.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

1. Change the route from `GET /venues/nearby` to `POST /venues/nearby`. Update the `app.get('/venues/nearby', ...)` call to `app.post('/venues/nearby', ...)`.
2. Update all `body()` validators to remain as `body()` — they now work correctly since the route accepts a POST body with `{ lat, lng, radius }`.
3. Install the `geofire-common` npm package if not already present: add it to `package.json` dependencies.
4. Import `geofire-common` at the top of `server.js`:

```
js
```

```
const geofire = require('geofire-common');
```

5. Replace the full Firestore table scan in the route handler with a GeoHash-based query:

```
js

const { lat, lng, radius } = req.body; // radius in km
const center = [parseFloat(lat), parseFloat(lng)];
const radiusInM = parseFloat(radius) * 1000;
const bounds = geofire.geohashQueryBounds(center, radiusInM);
const promises = bounds.map(b =>
  admin.firestore().collection('venues')
    .orderBy('geohash')
    .startAt(b[0])
    .endAt(b[1])
    .get()
);
const snapshots = await Promise.all(promises);
const venues = [];
snapshots.forEach(snap => {
  snap.docs.forEach(doc => {
    const data = doc.data();
    const distanceInKm = geofire.distanceBetween([data.lat, data.lng], center);
    if (distanceInKm <= parseFloat(radius)) {
      venues.push({ id: doc.id, ...data, distance: distanceInKm });
    }
  });
});
res.json({ venues });
```

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any other routes
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `POST /venues/nearby` with body `{ lat: 40.7128, lng: -74.0060, radius: 1 }` returns venues without a full collection scan.

2. `body()` validators on `lat`, `lng`, and `radius` now correctly reject requests missing those fields.
-

Task 7 — Move routes before `server.listen()`

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 1, Task 7.

PROBLEM

Push notification routes (`/notifications/register-token`, `/notifications/unregister-token`) and location/geofencing routes are defined in `server.js` after the `server.listen()` call (around line 280). Routes defined after `listen()` are registered too late and may not be reachable depending on the Node.js event loop timing. All route definitions must appear before `server.listen()`.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

1. Scan `server.js` for the `server.listen()` or `app.listen()` call. Note its line number.
2. Identify every `app.get()`, `app.post()`, `app.put()`, `app.delete()`, `app.patch()`, and `app.use()` call that appears after the `listen()` call.
3. Cut all of those route/middleware definitions and paste them immediately above the `listen()` call, preserving their relative order.
4. The `listen()` call must be the last significant statement in the file (error handler middleware may follow it if that was already the pattern, but no routes).
5. Do not reorder routes relative to each other — only move them as a block above `listen()`.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change the logic of any route — only their position in the file
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. Every `app.get/post/put/delete/patch/use` call appears before `server.listen()` or `app.listen()` in the file.
 2. `node server.js` starts without errors and all routes respond correctly.
-

Task 8 — Replace Storage `makePublic()` with signed URLs

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 1, Task 8.

PROBLEM

In the `POST /peeps` route in `server.js`, uploaded photos are made publicly accessible using `file.makePublic()` and `file.publicUrl()`. This makes every uploaded file permanently public with no access control, which is a security and billing risk. Replace this with a long-lived signed URL for now (with a note to implement short-lived rotation later).

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

1. In `server.js`, find the `POST /peeps` route handler, specifically the photo upload block.
2. Remove the following two lines (or their functional equivalents):

```
js

await file.makePublic();
const photoUrl = file.publicUrl();
```

3. Replace them with:

```
js

const [signedUrl] = await file.getSignedUrl({
  action: 'read',
  expires: '03-01-2500',
});
const photoUrl = signedUrl;
```

4. Add a comment above this block:

```
js  
  
// TODO: Replace long-lived signed URL with short-lived URL rotation before produ
```

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any other part of the POST /peeps route
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `server.js` contains no remaining calls to `makePublic()` or `publicUrl()`.
 2. Uploaded photos receive a signed URL string that begins with `https://storage.googleapis.com`.
-

Task 9 — Add per-route auth rate limiting

CONTEXT

Peep! is a real-time crowd intelligence mobile app. This is Phase 1, Task 9.

PROBLEM

The global rate limiter in `server.js` allows 100 requests per 15-minute window, which is appropriate for general API use but far too permissive for authentication endpoints.

`/auth/login` and `/auth/register` need a tighter, dedicated limiter to prevent brute-force attacks.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

1. In `server.js`, find where the global `rateLimit` middleware is defined (likely assigned to a variable such as `limiter` or `apiLimiter`).

2. Directly below it, add a new auth-specific limiter:

```
js

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 10,
  message: { error: 'Too many authentication attempts. Please try again in 15 min', },
  standardHeaders: true,
  legacyHeaders: false,
});
```

3. Find the `/auth/login` route definition. Add `authLimiter` as middleware before the route handler:

```
js

app.post('/auth/login', authLimiter, [...existing validators...], async (req, res) {
```

4. Find the `/auth/register` route definition. Add `authLimiter` the same way:

```
js

app.post('/auth/register', authLimiter, [...existing validators...], async (req, res) {
```

5. Do not apply `authLimiter` globally — only to these two routes.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change the global rate limiter
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `server.js` contains an `authLimiter` variable with `max: 10`.
2. Both `/auth/login` and `/auth/register` route definitions include `authLimiter` as a middleware argument.

Task 10 — Fix deprecated Android permissions

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 1, Task 10.

PROBLEM

`App.tsx` requests `READ_EXTERNAL_STORAGE` and `WRITE_EXTERNAL_STORAGE` permissions via `requestMultiple()`. These permissions are deprecated on Android 13+ (API level 33+) and will be rejected. The fix is to add version-gated logic: on Android 13 and above, request `READ_MEDIA_IMAGES` instead; on older versions, keep the existing permissions.

FILES TO EDIT

- `PeepMobile/App.tsx`

EXACT CHANGES REQUIRED

1. In `App.tsx`, find the `PermissionsAndroid.requestMultiple()` call.
2. Remove `PermissionsAndroid.PERMISSIONS.READ_EXTERNAL_STORAGE` and `PermissionsAndroid.PERMISSIONS.WRITE_EXTERNAL_STORAGE` from the permissions array.
3. Replace the entire permissions request block with version-gated logic:

ts

```
import { Platform, PermissionsAndroid } from 'react-native';

const requestAndroidPermissions = async () => {
  if (Platform.OS !== 'android') return;

  if (Platform.Version >= 33) {
    await PermissionsAndroid.request(
      PermissionsAndroid.PERMISSIONS.READ_MEDIA_IMAGES,
      {
        title: 'Photo Access',
        message: 'Peep1 needs access to your photos to post peeps.',
        buttonPositive: 'Allow',
      }
    );
  } else {
    await PermissionsAndroid.requestMultiple([
      PermissionsAndroid.PERMISSIONS.READ_EXTERNAL_STORAGE,
      PermissionsAndroid.PERMISSIONS.WRITE_EXTERNAL_STORAGE,
    ]);
  }
};
```

4. Call `requestAndroidPermissions()` where the old permission request was called.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any iOS permission handling
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `App.tsx` no longer unconditionally requests `READ_EXTERNAL_STORAGE` or `WRITE_EXTERNAL_STORAGE`.
2. `Platform.Version >= 33` branch requests `READ_MEDIA_IMAGES`; the else branch requests the legacy permissions.

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 1, Task 11.

PROBLEM

Multiple files use `any` types in places where proper TypeScript types exist or can be defined. Specifically: `navigation: any` should use `NativeStackNavigationProp`, `route.params as any` should use typed route params, and `handleRegister(userData: any)` should use the `RegisterData` type that already exists in `User.ts`.

FILES TO EDIT

- `PeepIMobile/App.tsx`
- All screen files (every file in `PeepIMobile/src/screens/`)
- `PeepIMobile/src/services/AuthService.ts`

EXACT CHANGES REQUIRED

1. In `App.tsx` and all screen files, find every prop typed as `navigation: any`. Replace with the proper React Navigation type:

```
ts

import { NativeStackNavigationProp } from '@react-navigation/native-stack';
// Define or import your RootStackParamList
type Props = {
  navigation: NativeStackNavigationProp<RootStackParamList, 'ScreenName'>;
};
```

Use the correct screen name for each file.

2. In all screen files, find every `route.params as any`. Replace with the typed params from `RootStackParamList`:

```
ts

import { RouteProp } from '@react-navigation/native';
type Props = {
  route: RouteProp<RootStackParamList, 'ScreenName'>;
};
const params = route.params; // now typed
```

3. In `AuthService.ts`, find `handleRegister(userData: any)`. Change to:

```
ts

import { RegisterData } from '../types/User';
handleRegister(userData: RegisterData): Promise<void>
```

4. If `RootStackParamList` does not exist, create it in `Peep1Mobile/src/types/Navigation.ts` and define all current screen param types.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any runtime logic — only type annotations
- Maintain existing code style and conventions
- After changes, the app must still compile with `tsc --noEmit` reporting zero `any`-related errors in the listed files

VERIFICATION

1. `tsc --noEmit` produces no errors related to `any` in `App.tsx`, screen files, or `AuthService.ts`.
2. `RootStackParamList` is defined and all screens reference it for their navigation and route types.

Task 12 — Extract `calculateDistance()` to shared utility

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 1, Task 12.

PROBLEM

The Haversine distance formula is copy-pasted verbatim in both `server.js` and `GeofencingService.js`. This duplication means any bug fix or improvement must be made in two places. Extract it into a single shared utility file.

FILES TO EDIT

- `server.js`

- `GeofencingService.js`
- `src/utils/geo.js` — CREATE NEW FILE

EXACT CHANGES REQUIRED

1. CREATE NEW FILE `src/utils/geo.js` with the following content:

```
js

/**
 * Calculate distance between two lat/lng coordinates using the Haversine formula
 * @param {number} lat1
 * @param {number} lng1
 * @param {number} lat2
 * @param {number} lng2
 * @returns {number} Distance in kilometers
 */
function calculateDistance(lat1, lng1, lat2, lng2) {
  const R = 6371; // Earth's radius in km
  const dLat = (lat2 - lat1) * Math.PI / 180;
  const dLng = (lng2 - lng1) * Math.PI / 180;
  const a =
    Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(lat1 * Math.PI / 180) * Math.cos(lat2 * Math.PI / 180) *
    Math.sin(dLng / 2) * Math.sin(dLng / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c;
}

module.exports = { calculateDistance };
```

(Copy the exact Haversine implementation from the existing files — do not rewrite it, just consolidate.)

2. In `server.js`, add at the top with other requires:

```
js

const { calculateDistance } = require('./src/utils/geo');
```

Then delete the local `calculateDistance` function definition from `server.js`.

3. In `GeofencingService.js`, add at the top:

```
js
```

```
const { calculateDistance } = require('./src/utils/geo');
```

Then delete the local `calculateDistance` function definition from `GeofencingService.js`.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change the logic of the Haversine function — only its location
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `src/utils/geo.js` exists and exports `calculateDistance`.
 2. Neither `server.js` nor `GeofencingService.js` defines a local `calculateDistance` function.
-

Task 13 — Wire up all commented-out API calls

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 1, Task 13.

PROBLEM

Every screen in the mobile app has its actual API calls commented out and replaced with mock data or `Alert('coming soon')`. A centralized `ApiService.ts` must be created to wrap all backend calls with proper error handling, and each screen must be wired to use it.

FILES TO EDIT

- `PeepMobile/src/services/ApiService.ts` — CREATE NEW FILE
- `PeepMobile/src/screens/VenueListScreen.tsx`
- `PeepMobile/src/screens/CreatePeepScreen.tsx`
- `PeepMobile/src/screens/VenueScreen.tsx`
- `PeepMobile/src/screens/MapScreen.tsx`
- `PeepMobile/src/screens/ProfileScreen.tsx`

EXACT CHANGES REQUIRED

1. CREATE NEW FILE `Peep1Mobile/src/services/ApiService.ts`:

```
ts

import { authService } from './AuthService';

const BASE_URL = process.env.API_BASE_URL || 'http://localhost:3000';

async function request<T>(method: string, path: string, body?: object): Promise<T> {
  const token = await authService.getIdToken();
  const response = await fetch(`${BASE_URL}${path}`, {
    method,
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
    },
    body: body ? JSON.stringify(body) : undefined,
  });
  if (!response.ok) {
    const error = await response.json().catch(() => ({ error: 'Unknown error' }));
    throw new Error(error.error || `HTTP ${response.status}`);
  }
  return response.json();
}

export const ApiService = {
  getNearbyVenues: (lat: number, lng: number, radius: number) =>
    request('POST', '/venues/nearby', { lat, lng, radius }),
  getVenue: (venueId: string) =>
    request('GET', `/venues/${venueId}`),
  createPeep: (data: object) =>
    request('POST', '/peeps', data),
  getFeed: () =>
    request('GET', '/feed'),
  getUserProfile: (userId: string) =>
    request('GET', `/users/${userId}`),
  updateUserProfile: (data: object) =>
    request('PUT', '/users/profile', data),
};
```

2. In each screen file listed, find all commented-out API calls and `Alert('coming soon')` blocks. Replace them with calls to the appropriate `ApiService` method. Wrap each call in try/catch and display errors using the existing error state pattern in each screen.
3. Remove all hardcoded mock data arrays that exist solely to substitute for real API calls.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change UI layout or styling in any screen
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `ApiService.ts` exists and exports all methods listed above.
 2. No screen file contains an `Alert('coming soon')` call related to data loading.
-

Task 14 — End-to-end validation gate

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 1, Task 14 — the validation gate before Phase 2 begins.

PROBLEM

This is not a code-writing task. It is a full end-to-end test that must pass before any Phase 2 work begins. Every fix from Tasks 1–13 must be verified working together in a real running environment.

FILES TO EDIT

- No files are changed in this task. This is a testing and validation task only.

EXACT CHANGES REQUIRED

1. Start the backend locally: `node server.js` (or `npm run dev` if a dev script exists). Confirm it starts without errors.
2. Start the React Native app in the iOS simulator: `npx react-native run-ios` (or `npm run ios`).

3. Execute the following flow completely without errors:
 - Open the app. Confirm the sign-up screen loads (not a crash, not a white screen).
 - Sign up a new user with a fresh email address.
 - Confirm the Firebase Auth user is created (check Firebase Console → Authentication).
 - Confirm the Firestore user document is created without a `password` field.
 - After sign-up, confirm the app navigates to the feed screen.
 - Create a new peep: tap the create button, fill in all required fields, submit.
 - Confirm the peep appears in the Firestore `peeps` collection.
 - Confirm the peep appears in the feed on the app.
4. If any step fails, fix the underlying bug (which belongs to one of Tasks 1-13), re-run the full flow from step 3, and do not proceed to Phase 2 until the entire flow passes.

CONSTRAINTS

- Do not write new features
- Do not modify any files unless fixing a regression from Tasks 1-13
- After this task is complete, commit with message: `[Phase 1 / Task 14] E2E validation gate passed`

VERIFICATION

1. Sign-up → token flow → create peep → peep visible in feed all work without errors.
2. No mock data, no `Alert('coming soon')`, no commented-out API calls remain in any screen involved in this flow.

PHASE 2 — Data Model & Core Features

Extend the data model to support the full Peepl crowd data spec. Build gamification, search, favorites, and social graph backend.

Task 15 — Extend Peep data model for full crowd data

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 2, Task 15. The current Peep data model is incomplete and must be extended to capture all crowd intelligence data the app

requires.

PROBLEM

The `Peep` TypeScript interface and Firestore schema are missing several crowd-specific fields. The `rating` field must be removed (PeepI uses crowd data, not star ratings). All new fields must be added to both the TypeScript interface and the backend route that accepts peep submissions.

FILES TO EDIT

- `PeepIMobile/src/types/Peep.ts`
- `server.js`

EXACT CHANGES REQUIRED

1. In `PeepIMobile/src/types/Peep.ts`, add the following fields to the `Peep` interface:

ts

```
crowdSize: 1 | 2 | 3 | 4 | 5;           // 1=Empty, 5=Packed
mfRatio: number;                       // 0-100, percentage male
akRatio: number;                       // 0-100, percentage adult
ageRanges: string[];                  // e.g. ['19-24', '25-30']
vibe: string[];                       // e.g. ['Chill', 'Dancing']
crowdTrend: 'getting_busier' | 'steady' | 'clearing_out';
```

2. In `PeepIMobile/src/types/Peep.ts`, remove the `rating` field from the `Peep` interface entirely.
3. Find the `CreatePeepData` interface (in `Peep.ts` or wherever it is defined). Add the same new fields and remove `rating` from it.
4. In `server.js`, find the `POST /peeps` route. Add `body()` validators for each new field:

js

```
body('crowdSize').isInt({ min: 1, max: 5 }),
body('mfRatio').isFloat({ min: 0, max: 100 }),
body('akRatio').isFloat({ min: 0, max: 100 }),
body('ageRanges').isArray(),
body('vibe').isArray(),
body('crowdTrend').isIn(['getting_busier', 'steady', 'clearing_out']),
```


5. In the `POST /peeps` Firestore write, add all new fields from `req.body` to the document being saved. Remove any `rating` field from the write.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any other Firestore routes
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `tsc --noEmit` passes with no type errors in `Peep.ts`.
 2. `POST /peeps` with a body containing all new fields saves them to Firestore without error.
-

Task 16 — Build full Peep creation screen

CONTEXT

Peopl is a real-time crowd intelligence mobile app. This is Phase 2, Task 16. The Peep creation screen must be rebuilt to match the 62-screen spec.

PROBLEM

The current `CreatePeepScreen.tsx` is incomplete. It must be fully rebuilt to capture all crowd data fields defined in Task 15, match the visual spec, and wire to the real backend.

FILES TO EDIT

- `PeoplMobile/src/screens/CreatePeepScreen.tsx`

EXACT CHANGES REQUIRED

Rebuild `CreatePeepScreen.tsx` with the following UI sections in a `ScrollView`:

1. **Venue row** — venue name auto-detected from GPS (from `LocationService`), with an "Override" button that opens a text input.
2. **Crowd Size slider** — horizontal slider from 1-5 with labels: Empty / Light / Moderate / Busy / Packed. Maps to `crowdSize` field.
3. **M/F ratio slider** — 0-100 with labels "All Female" at 0 and "All Male" at 100. Maps to `mfRatio`.

4. **Adult/Kid ratio slider** — 0–100 with labels "All Kids" at 0 and "All Adults" at 100. Maps to `akRatio`.
5. **Age ranges multi-select grid** — tappable chips in a `Wrap` layout. Options: `0-10`, `10-14`, `14-18`, `19-24`, `25-30`, `31-40`, `41-50`, `51-60`, `61-70`, `71-80`, `80+`. Multiple selections allowed. Maps to `ageRanges`.
6. **Vibe multi-select grid** — tappable chips. Options: `Chill`, `Laid Back`, `Casual`, `Rowdy`, `Dancing`, `Groovy`, `Formal`, `Sexy`, `Sports Fans`, `Families`, `Grungy`, `Business`. Multiple selections allowed. Maps to `vibe`.
7. **Crowd trend selector** — three segmented buttons: `Getting Busier`, `Steady`, `Clearing Out`. Maps to `crowdTrend`.
8. **Notes textarea** — optional free text, max 280 characters. Maps to `notes`.
9. **Photo/Video picker** — button that opens the device image picker. Shows thumbnail preview after selection.
10. **Submit button** — calls `ApiService.createPeep(data)` with all fields. On success: if response includes `isPioneer: true`, navigate to `PioneerCongrats` screen; otherwise navigate back to feed.

Use the existing blue (`#1565C0`) and gold (`#FFC107`) color scheme consistent with the rest of the app.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change `ApiService.ts` or any service files
- Maintain the existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. All 10 UI sections render in the simulator without layout errors.
 2. Submitting the form calls `ApiService.createPeep()` with all fields including `crowdSize`, `mfRatio`, `akRatio`, `ageRanges`, `vibe`, `crowdTrend`.
-

Task 17 — Build Pioneer detection system

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 2, Task 17.

PROBLEM

When a user submits a peep, the backend must check if they are the first person ever to peep at that venue. If so, they earn "Pioneer" status for that venue. This check must happen server-side in the `POST /peeps` route.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

1. In the `POST /peeps` route handler in `server.js`, after the new peep document is saved to Firestore, add the following logic:

```
js
```

```
// Pioneer detection
const venueId = req.body.venueId;
const existingPeeps = await admin.firestore()
  .collection('peeps')
  .where('venueId', '==', venueId)
  .limit(2) // only need to know if more than 1 exists
  .get();

const isPioneer = existingPeeps.docs.length === 1; // only the one we just created

if (isPioneer) {
  // Mark the peep as pioneer
  await peepRef.update({ isPioneer: true });

  // Update the venue document
  await admin.firestore().collection('venues').doc(venueId).update({
    pioneeredBy: req.user.uid,
    pioneeredAt: admin.firestore.FieldValue.serverTimestamp(),
  });

  // Award pioneer points (50 pts) – will integrate with PointsService in Task 18
  await admin.firestore().collection('users').doc(req.user.uid).update({
    points: admin.firestore.FieldValue.increment(50),
  });
}

res.json({ peep: { id: peepRef.id, ...peepData }, isPioneer });
```

2. Ensure the `peepRef` variable is assigned from the Firestore `add()` call result (i.e., `const peepRef = await admin.firestore().collection('peeps').add(peepData)`).
3. Ensure the response includes `isPioneer` as a boolean field.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any other routes
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. Creating the first peep for a venue returns `{ isPioneer: true }` in the API response.
 2. The venue document in Firestore is updated with `pioneeredBy` and `pioneeredAt` fields.
-

Task 18 — Build points and gamification system

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 2, Task 18.

PROBLEM

PeepI has a points system but no centralized service to manage it. Points are awarded for various actions and must be consistently applied. A `PointsService.js` must be created and wired into all relevant backend routes.

FILES TO EDIT

- `src/services/PointsService.js` — CREATE NEW FILE
- `server.js`

EXACT CHANGES REQUIRED

1. CREATE NEW FILE `src/services/PointsService.js`:

```
js
```

```

const admin = require('firebase-admin');

const POINTS_RULES = {
  PEEP_CREATED: 10,
  PIONEER_DISCOVERY: 50,
  LIKE_RECEIVED: 2,
  PEEP_WITH_PHOTO: 15,
  PEEP_FULLY_COMPLETE: 20,
};

async function awardPoints(userId, reason, amount) {
  const db = admin.firestore();
  const userRef = db.collection('users').doc(userId);
  const pointsLogRef = db.collection('points').doc();

  await db.runTransaction(async (transaction) => {
    transaction.update(userRef, {
      points: admin.firestore.FieldValue.increment(amount),
    });
    transaction.set(pointsLogRef, {
      userId,
      reason,
      amount,
      timestamp: admin.firestore.FieldValue.serverTimestamp(),
    });
  });
}

module.exports = { awardPoints, POINTS_RULES };

```

2. In `server.js`, add at the top:

```

js

const { awardPoints, POINTS_RULES } = require('./src/services/PointsService');

```

3. In the `POST /peeps` route, replace any direct `points: FieldValue.increment(X)` calls with:

```

js

```

```

await awardPoints(req.user.uid, 'PEEP_CREATED', POINTS_RULES.PEEP_CREATED);
if (hasPhoto) await awardPoints(req.user.uid, 'PEEP_WITH_PHOTO', POINTS_RULES.PEEP_WITH_PHOTO);
if (isFullyComplete) await awardPoints(req.user.uid, 'PEEP_FULLY_COMPLETE', POINTS_RULES.PEEP_FULLY_COMPLETE);
if (isPioneer) await awardPoints(req.user.uid, 'PIONEER_DISCOVERY', POINTS_RULES.PIONEER_DISCOVERY);

```

Define `isFullyComplete` as: all optional fields (ageRanges, vibe, crowdTrend, notes, photo) are present.

4. Add a new route in `server.js` before `listen()`:

```

js

app.get('/users/points', authenticate, async (req, res) => {
  try {
    const userDoc = await admin.firestore().collection('users').doc(req.user.uid)
    const points = userDoc.data()?.points || 0;
    const history = await admin.firestore()
      .collection('points')
      .where('userId', '==', req.user.uid)
      .orderBy('timestamp', 'desc')
      .limit(20)
      .get();
    const log = history.docs.map(d => ({ id: d.id, ...d.data() }));
    res.json({ points, log });
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});

```

CONSTRAINTS

- Do not modify any files not listed above
- Do not change the like route yet — Task 21 handles that
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `src/services/PointsService.js` exists and exports `awardPoints` and `POINTS_RULES`.

2. `GET /users/points` returns `{ points: number, log: array }` for an authenticated user.
-

Task 19 — Build leaderboard API

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 2, Task 19.

PROBLEM

The leaderboard feature requires three API endpoints that don't yet exist: global ranking, friends ranking, and current user's rank.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

Add the following three routes to `server.js` before `listen()`:

1. **Global leaderboard:**

js


```
app.get('/leaderboard/everyone', authenticate, async (req, res) => {
  try {
    const snapshot = await admin.firestore()
      .collection('users')
      .orderBy('points', 'desc')
      .limit(50)
      .get();
    const users = snapshot.docs.map((d, i) => ({
      rank: i + 1,
      userId: d.id,
      username: d.data().username,
      points: d.data().points || 0,
      pioneerCount: d.data().pioneerCount || 0,
      profileImageUrl: d.data().profileImageUrl || null,
    }));
    res.json({ leaderboard: users });
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});
```

2. Friends leaderboard:

js

```

app.get('/leaderboard/friends', authenticate, async (req, res) => {
  try {
    const followingSnap = await admin.firestore()
      .collection('follows')
      .where('followerId', '==', req.user.uid)
      .get();
    const followingIds = [req.user.uid, ...followingSnap.docs.map(d => d.data().f
    // Firestore 'in' operator supports max 10 – chunk if needed
    const chunks = [];
    for (let i = 0; i < followingIds.length; i += 10) {
      chunks.push(followingIds.slice(i, i + 10));
    }
    const userDocs = (await Promise.all(
      chunks.map(chunk =>
        admin.firestore().collection('users').where(admin.firestore.FieldPath.doc
      )
    )).flatMap(s => s.docs);
    const users = userDocs
      .sort((a, b) => (b.data().points || 0) - (a.data().points || 0))
      .map((d, i) => ({
        rank: i + 1,
        userId: d.id,
        username: d.data().username,
        points: d.data().points || 0,
        pioneerCount: d.data().pioneerCount || 0,
        profileImageUrl: d.data().profileImageUrl || null,
      }));
    res.json({ leaderboard: users });
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});

```

3. Current user's rank:

js

```

app.get('/leaderboard/rank', authenticate, async (req, res) => {
  try {
    const userDoc = await admin.firestore().collection('users').doc(req.user.uid)
    const myPoints = userDoc.data()?.points || 0;
    const rankSnap = await admin.firestore()
      .collection('users')
      .where('points', '>', myPoints)
      .get();
    const rank = rankSnap.size + 1;
    res.json({ rank, points: myPoints });
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});

```

4. In the Firebase Console (or via a migration script), create a composite Firestore index on `users` collection: field `points` descending.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any existing routes
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `GET /leaderboard/everyone` returns an array of up to 50 users ordered by points descending.
2. `GET /leaderboard/rank` returns `{ rank: number, points: number }` for the authenticated user.

Task 20 — Build venue page with crowd history

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 2, Task 20.

PROBLEM

`VenueScreen.tsx` shows basic venue info but is not wired to real data and lacks the crowd history sparkline and peep feed required by the spec.

FILES TO EDIT

- `PeepMobile/src/screens/VenueScreen.tsx`

EXACT CHANGES REQUIRED

Replace the existing `VenueScreen.tsx` content with a fully wired screen:

1. **Hero image** — full-width venue photo from `venue.imageUrl`. Use `ImageBackground` with a gradient scrim overlay containing venue name.
2. **Current crowd card** — aggregate from the most recent peep for this venue: `crowdSize`, vibe tags, `crowdTrend` indicator, timestamp of last peep.
3. **12-hour sparkline** — fetch from `GET /venues/:venueId/crowd-history` (built in Task 71 — for now, stub the endpoint to return 12 zero data points). Render a simple SVG line chart using `react-native-svg`. X-axis: hours (12h to now). Y-axis: `crowdSize` 1–5. Color the line blue (`#1565C0`).
4. **Peeps feed** — `FlatList` of all peeps for this venue in reverse chronological order. Each peep row: reporter avatar + username, crowd pill (colored by level), vibe tags, timestamp, Like and Comment counts with tappable icons.
5. **Native deal injection** — after the 2nd peep in the list, if a merchant ad is active near this venue (check via `GET /merchant/feed?lat=&lng=`), inject a full-width deal card with merchant name, offer text, and gold Claim button.
6. Wire all data fetching using `ApiService`. Show loading state while fetching. Show empty state (Task 54 UI) if `venue.peepCount === 0`.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change `ApiService.ts` — add methods there only if absolutely necessary
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. `VenueScreen` loads real venue data from the API and renders without crashes.
2. The peep feed renders in reverse chronological order with like/comment counts.

Task 21 — Build likes and comments backend

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 2, Task 21.

PROBLEM

There are no backend endpoints for liking/unliking peeps or posting/reading comments. These must be built now so the feed and peep detail screens can be wired.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

Add the following routes to `server.js` before `listen()`:

1. **Like a peep:**

```
js
```

```

app.post('/peeps/:peepId/like', authenticate, async (req, res) => {
  const { peepId } = req.params;
  const userId = req.user.uid;
  const db = admin.firestore();
  const peepRef = db.collection('peeps').doc(peepId);
  const likeRef = peepRef.collection('likes').doc(userId);
  await db.runTransaction(async t => {
    const likeDoc = await t.get(likeRef);
    if (!likeDoc.exists) {
      t.set(likeRef, { likedAt: admin.firestore.FieldValue.serverTimestamp() });
      t.update(peepRef, { likeCount: admin.firestore.FieldValue.increment(1) });
    }
  });
  // Award 2 pts to peep author
  const peepDoc = await peepRef.get();
  if (peepDoc.exists) {
    await awardPoints(peepDoc.data().userId, 'LIKE_RECEIVED', POINTS_RULES.LIKE_R
  }
  res.json({ liked: true });
});

```

2. Unlike a peep:

```

js

app.delete('/peeps/:peepId/like', authenticate, async (req, res) => {
  const { peepId } = req.params;
  const userId = req.user.uid;
  const db = admin.firestore();
  const peepRef = db.collection('peeps').doc(peepId);
  const likeRef = peepRef.collection('likes').doc(userId);
  await db.runTransaction(async t => {
    const likeDoc = await t.get(likeRef);
    if (likeDoc.exists) {
      t.delete(likeRef);
      t.update(peepRef, { likeCount: admin.firestore.FieldValue.increment(-1) });
    }
  });
  res.json({ liked: false });
});

```

3. Post a comment:

```
js

app.post('/peeps/:peepId/comments', authenticate, async (req, res) => {
  const { peepId } = req.params;
  const { text } = req.body;
  if (!text || text.trim().length === 0) return res.status(400).json({ error: 'Co
  const db = admin.firestore();
  const peepRef = db.collection('peeps').doc(peepId);
  const commentRef = await peepRef.collection('comments').add({
    userId: req.user.uid,
    text: text.trim(),
    createdAt: admin.firestore.FieldValue.serverTimestamp(),
  });
  await peepRef.update({ commentCount: admin.firestore.FieldValue.increment(1) });
  res.json({ commentId: commentRef.id });
});
```

4. Get comments:

```
js

app.get('/peeps/:peepId/comments', authenticate, async (req, res) => {
  const { peepId } = req.params;
  const snap = await admin.firestore()
    .collection('peeps').doc(peepId)
    .collection('comments')
    .orderBy('createdAt', 'desc')
    .limit(50)
    .get();
  const comments = await Promise.all(snap.docs.map(async d => {
    const data = d.data();
    const userDoc = await admin.firestore().collection('users').doc(data.userId).
    return { id: d.id, ...data, username: userDoc.data()?.username || 'Unknown' }
  }));
  res.json({ comments });
});
```

5. Get likers:

js

```
app.get('/peeps/:peepId/likes', authenticate, async (req, res) => {
  const { peepId } = req.params;
  const snap = await admin.firestore()
    .collection('peeps').doc(peepId)
    .collection('likes')
    .limit(50)
    .get();
  res.json({ likers: snap.docs.map(d => d.id) });
});
```

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `POST /peeps/:peepId/like` increments `likeCount` on the peep document and creates a subcollection document.
 2. `GET /peeps/:peepId/comments` returns comments with username data included.
-

Task 22 — Build crowd confirmation push trigger

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 2, Task 22.

PROBLEM

When a venue gets busy, users who favorited that venue should receive a push notification. This logic must be triggered after every new peep is created and must be throttled to avoid notification spam.

FILES TO EDIT

- `src/services/NotificationService.js`

EXACT CHANGES REQUIRED

1. In `NotificationService.js`, add a new exported function `triggerCrowdConfirmationIfNeeded`:

```
js
```

```

async function triggerCrowdConfirmationIfNeeded(venueId, venueName) {
  const db = admin.firestore();
  const thirtyMinutesAgo = new Date(Date.now() - 30 * 60 * 1000);

  // Check: 3+ peeps at this venue in the last 30 minutes with crowdSize >= 4
  const recentPeeps = await db.collection('peeps')
    .where('venueId', '==', venueId)
    .where('crowdSize', '>=', 4)
    .where('createdAt', '>=', thirtyMinutesAgo)
    .get();

  if (recentPeeps.size < 3) return;

  // Get all users who favorited this venue
  const favoritesSnap = await db.collectionGroup('favorites')
    .where('venueId', '==', venueId)
    .get();

  const twoHoursAgo = new Date(Date.now() - 2 * 60 * 60 * 1000);

  for (const favDoc of favoritesSnap.docs) {
    const userId = favDoc.ref.parent.parent.id;

    // Throttle: check if we already sent this notification in the last 2 hours
    const recentNotifSnap = await db.collection('notification_throttle')
      .where('userId', '==', userId)
      .where('venueId', '==', venueId)
      .where('sentAt', '>=', twoHoursAgo)
      .limit(1)
      .get();

    if (!recentNotifSnap.empty) continue;

    // Get user's FCM token
    const userDoc = await db.collection('users').doc(userId).get();
    const fcmToken = userDoc.data()?.fcmToken;
    if (!fcmToken) continue;

    // Send push notification
    await admin.messaging().send({
      token: fcmToken,
      notification: {
        title: `${venueName} is getting crowded`,

```

```

        body: 'Busy right now.',
      },
      data: { venueId, type: 'crowd_spike' },
    });

    // Log throttle record
    await db.collection('notification_throttle').add({
      userId,
      venueId,
      sentAt: admin.firestore.FieldValue.serverTimestamp(),
    });
  }
}

module.exports = { ...(module.exports || {}), triggerCrowdConfirmationIfNeeded };

```

2. In `server.js`, import `triggerCrowdConfirmationIfNeeded` from `NotificationService.js`.
3. In the `POST /peeps` route, after saving the peep, add:

```

js

triggerCrowdConfirmationIfNeeded(venueId, venueName).catch(e =>
  logger.error('Push trigger failed:', e)
);

```

Call this fire-and-forget (do not await it in the request handler).

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `triggerCrowdConfirmationIfNeeded` is exported from `NotificationService.js`.
 2. The function is called (fire-and-forget) at the end of the `POST /peeps` route handler.
-

Task 23 — Build GetPeeps search backend

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 2, Task 23.

PROBLEM

The app needs search functionality for venues by name, by proximity, by city, and for users by username. None of these search endpoints exist yet.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

Add the following routes to `server.js` before `listen()`:

1. Text venue search:

```
js

app.get('/search/venues', authenticate, async (req, res) => {
  const { q } = req.query;
  if (!q || q.trim().length < 2) return res.status(400).json({ error: 'Query must'
  const term = q.toLowerCase().trim();
  const snap = await admin.firestore().collection('venues')
    .orderBy('nameLower')
    .startAt(term)
    .endAt(term + '\uf8ff')
    .limit(20)
    .get();
  const venues = snap.docs.map(d => ({ id: d.id, ...d.data() }));
  res.json({ venues });
});
```

2. Nearby venue search (GeoHash-based — uses geofire-common from Task 6):

```
js
```

```

app.get('/search/venues/nearby', authenticate, async (req, res) => {
  const { lat, lng, radius = 1 } = req.query;
  if (!lat || !lng) return res.status(400).json({ error: 'lat and lng required' });
  const center = [parseFloat(lat), parseFloat(lng)];
  const radiusInM = parseFloat(radius) * 1000;
  const bounds = geofire.geohashQueryBounds(center, radiusInM);
  const snaps = await Promise.all(
    bounds.map(b => admin.firestore().collection('venues').orderBy('geohash').startAt(b)
  );
  const venues = snaps.flatMap(s => s.docs).filter(d => {
    const { lat: vLat, lng: vLng } = d.data();
    return geofire.distanceBetween([vLat, vLng], center) <= parseFloat(radius);
  }).map(d => ({ id: d.id, ...d.data() }));
  res.json({ venues });
});

```

3. City venue search:

js

```

app.get('/search/venues/city', authenticate, async (req, res) => {
  const { city } = req.query;
  if (!city) return res.status(400).json({ error: 'city required' });
  const snap = await admin.firestore().collection('venues')
    .where('cityLower', '==', city.toLowerCase().trim())
    .limit(50)
    .get();
  res.json({ venues: snap.docs.map(d => ({ id: d.id, ...d.data() }))) });
});

```

4. User search:

js

```
app.get('/search/users', authenticate, async (req, res) => {
  const { q } = req.query;
  if (!q || q.trim().length < 2) return res.status(400).json({ error: 'Query must'
  const term = q.toLowerCase().trim();
  const snap = await admin.firestore().collection('users')
    .orderBy('usernameLower')
    .startAt(term)
    .endAt(term + '\uf8ff')
    .limit(20)
    .get();
  res.json({ users: snap.docs.map(d => ({ id: d.id, username: d.data().username,
  }));
```

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any existing routes
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `GET /search/venues?q=bar` returns venues whose names start with "bar".
 2. `GET /search/users?q=jo` returns users whose usernames start with "jo".
-

Task 24 — Build favorites system

CONTEXT

Peopl is a real-time crowd intelligence mobile app. This is Phase 2, Task 24.

PROBLEM

Users can't save favorite venues. The backend endpoints and the profile screen UI for favorites must both be built.

FILES TO EDIT

- `server.js`
- `PeoplMobile/src/screens/ProfileScreen.tsx`

EXACT CHANGES REQUIRED

1. Add the following routes to `server.js` before `listen()`: **Add favorite:**

```
js

app.post('/users/favorites/:venueId', authenticate, async (req, res) => {
  const { venueId } = req.params;
  await admin.firestore()
    .collection('users').doc(req.user.uid)
    .collection('favorites').doc(venueId)
    .set({ venueId, addedAt: admin.firestore.FieldValue.serverTimestamp() });
  res.json({ favorited: true });
});
```

Remove favorite:

```
js

app.delete('/users/favorites/:venueId', authenticate, async (req, res) => {
  const { venueId } = req.params;
  await admin.firestore()
    .collection('users').doc(req.user.uid)
    .collection('favorites').doc(venueId)
    .delete();
  res.json({ favorited: false });
});
```

List favorites with current crowd data:

```
js
```

```

app.get('/users/favorites', authenticate, async (req, res) => {
  const snap = await admin.firestore()
    .collection('users').doc(req.user.uid)
    .collection('favorites')
    .orderBy('addedAt', 'desc')
    .get();
  const favorites = await Promise.all(snap.docs.map(async d => {
    const venueDoc = await admin.firestore().collection('venues').doc(d.data().venueId).get();
    const latestPeep = await admin.firestore().collection('peeps')
      .where('venueId', '==', d.data().venueId)
      .orderBy('createdAt', 'desc')
      .limit(1)
      .get();
    return {
      venueId: d.data().venueId,
      ...venueDoc.data(),
      currentCrowdSize: latestPeep.docs[0]?.data()?.crowdSize || null,
      lastPeepAt: latestPeep.docs[0]?.data()?.createdAt || null,
    };
  }));
  res.json({ favorites });
});

```

2. In `ProfileScreen.tsx`, add a "Favorites" section that fetches from `GET /users/favorites` and displays venue names with current crowd data. Each venue row should have a heart icon that calls `DELETE /users/favorites/:venueId` to unfavorite inline.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `POST /users/favorites/:venueId` creates a document in the user's `favorites` subcollection.
2. `GET /users/favorites` returns the list of favorited venues with `currentCrowdSize` populated from the most recent peep.

PHASE 3 — All Missing Consumer Screens

Build every screen in the 62-screen spec. New screens are new files. Existing screens that need rebuilding are noted. All screens must match the 2025 visual language.

Task 25 — Splash / Onboarding screens (3 screens)

CONTEXT

Peep! is a real-time crowd intelligence mobile app. This is Phase 3, Task 25.

PROBLEM

There are no onboarding screens. First-time users see the feed immediately after sign-up with no context about the app. Three onboarding screens must be built and shown only on first launch.

FILES TO EDIT

- Peep!Mobile/src/screens/OnboardingScreen.tsx — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create OnboardingScreen.tsx with the following:

- Container:** Full-screen blue gradient background (#1565C0 to #0D47A1). Horizontal FlatList or ScrollView with pagingEnabled.
- Screen 1:** Large 🧑 emoji centered. Title: "Know before you go." in white bold 28pt. Subtitle: "See real crowd levels at any venue, updated by real people in real time." in white 16pt.
- Screen 2:** Large 📍 emoji. Title: "Peep anywhere. Anytime." Subtitle: "Post crowd updates in seconds and help your community know what's happening."
- Screen 3:** Large 🌟 emoji. Title: "Earn Pioneer status." Subtitle: "Be the first to peep a venue and claim your Pioneer badge forever."
- Navigation controls:** Row of 3 dot indicators (filled dot = current screen, outline dot = other). "Next" button (white, blue text) advances to next screen. "Skip" text button on screens 1 and 2. Screen 3 shows "Join Now ➔" button (gold #FFC107, black text) that navigates to Register.

6. **First-launch gate:** On mount, check `AsyncStorage.getItem('hasSeenOnboarding')`. If the value is `'true'`, skip this screen and navigate directly to `Login`. On "Join Now" tap, set `AsyncStorage.setItem('hasSeenOnboarding', 'true')` before navigating.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change the navigation stack yet — that is Task 73
- Maintain the existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. Three swipeable screens render correctly in the simulator with dot indicators.
 2. `AsyncStorage` is checked on mount and the screen only shows once per install.
-

Task 26 — Sign In screen redesign

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 3, Task 26.

PROBLEM

The current `LoginScreen.tsx` does not match the 2025 visual spec. It must be rebuilt.

FILES TO EDIT

- `PeeplMobile/src/screens/LoginScreen.tsx`

EXACT CHANGES REQUIRED

Rebuild `LoginScreen.tsx` with the following layout (top to bottom):

1. **Background:** Full-screen solid blue (`#1565C0`).
2. **Logo:** `"peep1"` in white, bold, 52pt, centered, with slight negative letter-spacing.
3. **Tagline:** Italic white text: `"Know before you go."` 16pt, centered.
4. **Email input:** White-background `TextInput`, rounded corners (`borderRadius: 12`), `placeholder="Email address"`, `keyboardType="email-address"`, `autoCapitalize="none"`.

5. **Password input:** Same styling, `placeholder="Password"`, `secureTextEntry`.
6. **Sign In button:** Full-width, white background, blue text (`#1565C0`), bold, "Sign In". Calls `authService.login({ email, password })` on press. Shows `ActivityIndicator` while loading.
7. **Divider:** "or" with horizontal lines on each side.
8. **Facebook button:** Full-width, dark blue background (`#1877F2`), white "Continue with Facebook" text. Shows `Alert('Facebook login coming soon')` for now.
9. **Google button:** Full-width, white background, dark text, "Continue with Google". Shows `Alert('Google login coming soon')` for now.
10. **Footer link:** "New to Peepl? Join now" — "Join now" in gold (`#FFC107`), tappable, navigates to `Register`.
11. **Error display:** Red text below the Sign In button for auth errors.

CONSTRAINTS

- Do not modify any files not listed above
- Do not implement actual OAuth — stubs with `Alert` are acceptable for this task
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. The screen renders with a full blue background, logo, and all input fields in the simulator.
 2. Submitting valid credentials calls `authService.login()` and navigates to the feed on success.
-

Task 27 — Sign Up screen redesign

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 3, Task 27.

PROBLEM

The current `RegisterScreen.tsx` does not match the 2025 visual spec. It must be rebuilt.

FILES TO EDIT

- `PeeplMobile/src/screens/RegisterScreen.tsx`

EXACT CHANGES REQUIRED

Rebuild `RegisterScreen.tsx` with the following layout inside a `ScrollView`:

1. **Background:** Full-screen solid blue (`#1565C0`).
2. **Header:** `"Join Peepl"` in white bold 32pt.
3. **OAuth buttons (top):**
 - Full-width Facebook button (`#1877F2`, white text `"Sign up with Facebook"`)
 - Full-width Google button (white background, dark text `"Sign up with Google"`)
 - Both show `Alert('OAuth coming soon')` for now.
4. **Divider:** `"or sign up with email"` with lines.
5. **Form inputs** (white background, `borderRadius: 12`):
 - Full Name (`placeholder="Full name"`)
 - Username (`placeholder="Username"`, `autoCapitalize="none"`)
 - Email (`placeholder="Email address"`, `keyboardType="email-address"`)
 - Password (`placeholder="Password"`, `secureTextEntry`, min 6 chars validation)
6. **Terms text:** `"By creating an account, you agree to our Terms of Service and Privacy Policy."` White text, 12pt, centered. `"Terms of Service"` and `"Privacy Policy"` are gold and tappable (navigate to a `WebView` with the URL for now, stub the URL).
7. **Create Account button:** Full-width gold (`#FFC107`), black text, bold. Calls `authService.register({ fullName, username, email, password })` on press. Shows `ActivityIndicator` while loading. On success, navigates to `SignUpConfirmed`.
8. **Sign in link:** `"Already have an account? Sign in"` at bottom.

CONSTRAINTS

- Do not modify any files not listed above
- Do not implement actual OAuth
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. All form inputs render and are fillable in the simulator.

2. Submitting valid data calls `authService.register()` and navigates to `SignUpConfirmed` on success.
-

Task 28 — Sign Up Confirmed screen

CONTEXT

Peep! is a real-time crowd intelligence mobile app. This is Phase 3, Task 28.

PROBLEM

After a successful sign-up, users land on the feed with no celebration or guidance. A dedicated confirmation screen is needed.

FILES TO EDIT

- `Peep!Mobile/src/screens/SignUpConfirmedScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `SignUpConfirmedScreen.tsx`:

1. **Background:** Full-screen blue gradient (`#1565C0` to `#0D47A1`).
2. **Emoji:** Large 🎉 centered at the top third of the screen.
3. **Title:** `"Welcome to Peep!"` white bold 28pt, centered.
4. **User name line:** The user's display name (from Firebase Auth `currentUser.displayName`) in gold (`#FFC107`) bold 22pt, centered.
5. **Onboarding steps list** (3 rows with checkboxes/bullet icons):
 - ✓ Check the feed to see what's happening near you
 - ✓ Post your first Peep to earn your first points
 - ✓ Earn Pioneer status at your first venue Each row: white check circle icon on the left, white text on the right.
6. **CTA button:** Full-width gold button `"Take me to the feed →"` that navigates to the main Feed screen. Replace the navigation stack so the user cannot go back to this screen.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change the navigation stack architecture — that is Task 73
- Maintain existing blue/gold color scheme

- After changes, the app must still compile

VERIFICATION

1. The screen renders with the user's display name in gold.
 2. Tapping the CTA navigates to the feed and cannot be navigated back to via the back button.
-

Task 29 — Location & Push permission screens

CONTEXT

Peep! is a real-time crowd intelligence mobile app. This is Phase 3, Task 29.

PROBLEM

After sign-up confirmation, users must be asked for location and push notification permissions. These must be presented as onboarding screens with clear benefit explanations before triggering the OS permission dialogs.

FILES TO EDIT

- `Peep!Mobile/src/screens/PermissionsScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `PermissionsScreen.tsx` that handles both permissions sequentially:

1. **State:** `currentStep: 'location' | 'push'`. Starts with `'location'`.
2. **Location step:**
 - Full blue background.
 - Large  icon centered.
 - Title: `"Enable location"` white bold 24pt.
 - Three benefit bullets (checkmarks):
 - `"See crowd levels at venues near you"`
 - `"Auto-detect your venue when posting"`
 - `"Get notified about your favorite spots"`
 - `"Allow Location"` gold button → calls `Geolocation.requestAuthorization()` (iOS) or `PermissionsAndroid.request(ACCESS_FINE_LOCATION)` (Android) → on resolution (allowed or denied), transition to `'push'` step.

- `"Not now"` white text link → skips to `'push'` step.

3. Push step:

- Same blue background.
- Large 🔔 icon.
- Title: `"Stay in the loop"`.
- Three notification type examples:
 - `"🔥 Central Park is getting packed"`
 - `"⭐ You earned Pioneer status!"`
 - `"❤️ Someone liked your Peep"`
- `"Allow Notifications"` gold button → calls `messaging().requestPermission()` from `@react-native-firebase/messaging` → on resolution, navigate to Feed.
- `"Maybe later"` link → navigates to Feed.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. Both steps render correctly and transition between each other.
 2. Tapping `"Allow Location"` triggers the OS location permission dialog on device.
-

Task 30 — Pioneer Congrats screen

CONTEXT

Peep! is a real-time crowd intelligence mobile app. This is Phase 3, Task 30.

PROBLEM

When a user earns Pioneer status after submitting a peep (API returns `isPioneer: true`), there is no celebration screen. This screen must be built and triggered at the right moment.

FILES TO EDIT

- `Peep!Mobile/src/screens/PioneerCongratScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `PioneerCongratScreen.tsx`:

1. **Background:** Full-screen blue gradient.
2. **Large** 🌟 centered, 80pt.
3. **"Pioneer" gold badge** — pill-shaped container ((🟡 #FFC107) background, (#000) text), bold, centered below the star.
4. **Title:** ("You're a Peep1 Pioneer!") white bold 28pt, centered.
5. **Venue name:** `route.params.venueName` displayed in gold bold 20pt, centered.
6. **Three stats row:**
 - "1st Pioneer" (always)
 - "X Venues Pioneered" — from `route.params.venuesPioneeredCount`
 - "+50 pts" for this discovery Each stat as a card with a number on top and label below, white text.
7. **Two buttons:**
 - "Back to Feed" (white, blue text) — navigates to Feed screen, clears modal stack.
 - "Share your Pioneer status" (outlined white border, white text) — triggers native share sheet with text: "I just became the first Pioneer at [venueName] on Peep1! 🌟 #Peep1Pioneer".

Route params type:

```
ts

type PioneerCongratParams = {
  venueName: string;
  venuesPioneeredCount: number;
};
```

In `CreatePeepScreen.tsx`, after a successful peep submission where `response.isPioneer === true`, navigate to `PioneerCongrat` with the correct params.

CONSTRAINTS

- Do not modify any files not listed above except `CreatePeepScreen.tsx` to add the navigation trigger
- Maintain existing blue/gold color scheme

- After changes, the app must still compile

VERIFICATION

1. `PioneerCongratScreen` renders with the venue name in gold and all three stats visible.
 2. `"Share your Pioneer status"` opens the native share sheet with the correct text.
-

Task 31 — Deals screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 31.

PROBLEM

There is no Deals screen. Merchants run time-limited deals and they must be surfaced to consumers in a dedicated screen.

FILES TO EDIT

- `PeepIMobile/src/screens/DealsScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `DealsScreen.tsx`:

1. **Header row:** `"Deals"` title left-aligned, `"Advertise"` button right-aligned (tappable, navigates to `MerchantSignIn`).
2. **Filter chips row** (horizontal `ScrollView`): `All`, `Food`, `Drinks`, `Services`, `Events`.
Active chip: gold background, black text. Inactive: grey background. Tapping a chip filters the list below.
3. **Deals list** (`FlatList`): Each card is full-width with:
 - Full-bleed photo background (`height: 200`).
 - Gradient scrim (transparent to black, bottom half).
 - Over the scrim (bottom-left): merchant name (white bold 18pt), offer text (white 14pt), distance from user (white 12pt).
 - Top-right: countdown timer (`"Ends in HH:MM:SS"`) updated every second via `setInterval`).
 - Bottom-right: gold `"Claim"` button — on tap, shows a confirmation modal then navigates to `DealClaimed` screen with deal data as params.

4. **Data:** Fetch from `GET /merchant/feed?lat=&lng=` using the user's current coordinates. Show `ActivityIndicator` while loading. Show `"No deals near you right now"` empty state if list is empty.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The screen renders with filter chips and at least the empty state visible.
 2. Filter chips filter the displayed deals by category.
-

Task 32 — Deal Claimed confirmation screen

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 3, Task 32.

PROBLEM

After claiming a deal, users need a screen to show their server to redeem it. This screen must keep the device screen awake and display a countdown.

FILES TO EDIT

- `PeeplMobile/src/screens/DealClaimedScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `DealClaimedScreen.tsx`:

1. **Route params:** `{ deal: { merchantName, offerText, expiresAt: string (ISO), venueImageUrl, adId } }`.
2. **Hero image:** Full-width venue photo from `deal.venueImageUrl` at the top, height 200.
3. **Gold stamp overlay on the image:** Centered `"DEAL CLAIMED ✓"` in a gold rotated rectangle (like a physical stamp), bold 24pt black text.
4. **Merchant name:** `deal.merchantName` centered, white bold 20pt, on blue background.

5. **Offer text:** `deal.offerText` centered, white 16pt.
6. **"Show this screen to your server"** **instruction** in italic gold text, centered.
7. **Countdown timer:** `"Expires in HH:MM:SS"` — counts down from `deal.expiresAt`. Uses `setInterval` updated every second. When timer reaches 0, display `"This deal has expired"` in red.
8. **"Show staff this screen"** **button** (gold, full-width) — calls `KeepAwake.activate()` from `react-native-keep-awake` to prevent screen timeout.
9. On `componentWillUnmount`, call `KeepAwake.deactivate()`.
10. On mount, call `POST /merchant/ads/:adId/claim` to record the claim.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The countdown timer updates every second correctly.
 2. `POST /merchant/ads/:adId/claim` is called on mount.
-

Task 33 — Get Peeps screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 33.

PROBLEM

There is no "Get Peeps" screen — a discovery screen for finding venue crowd data by location, city, or specific venue search. This is a core consumer feature.

FILES TO EDIT

- `PeepIMobile/src/screens/GetPeepsScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `GetPeepsScreen.tsx`:

1. **Three mode tabs** (segmented control at top): `Near Me`, `New City`, `Specific Venue`.

2. **Near Me tab:**

- Auto-fetches trending venues using `GET /search/venues/nearby` with user's current coordinates and radius 5km.
- Renders results as full-width photo cards (each 180px tall): venue photo background, gradient scrim, venue name (white bold 20pt), crowd pill (colored circle with `crowdSize` number).
- Tapping a card navigates to `VenueScreen` with `venueId`.

3. **New City tab:**

- Text input: `placeholder="Enter a city"`.
- `"Search"` button triggers `GET /search/venues/city?city=term`.
- Results in the same full-width photo card format.

4. **Specific Venue tab:**

- Real-time text input (debounced 300ms) that calls `GET /search/venues?q=term` on each keystroke.
- Results as a list (not photo cards): venue name, category, distance.
- Tapping navigates to `VenueScreen`.

5. Show `ActivityIndicator` while loading in all tabs. Show empty state text when no results.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. All three tabs render and are switchable without crashes.
2. The Near Me tab fetches real venues from the backend using the user's GPS coordinates.

Task 34 — Map screen full rebuild

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 3, Task 34.

PROBLEM

The existing `MapScreen.tsx` shows a basic map with simple markers. It must be rebuilt to show crowd heat zones, color-coded crowd pins, deal pins, and filter controls.

FILES TO EDIT

- `Peep1Mobile/src/screens/MapScreen.tsx`

EXACT CHANGES REQUIRED

Rebuild `MapScreen.tsx`:

- Map:** Use `react-native-maps` `MapView` covering the full screen, centered on user's location.
- User location dot:** `showsUserLocation={true}` on `MapView` plus a custom pulsing ring overlay at user's coordinates (animate opacity $0.2 \rightarrow 0.6 \rightarrow 0.2$ with 2s loop using `Animated`).
- Crowd heat zones:** For each venue with a recent peep, render a `Circle` on the map:
 - Radius: 100m.
 - Color based on `crowdSize`: 1-2 = `rgba(76,175,80,0.3)` (green), 3 = `rgba(255,167,38,0.3)` (amber), 4-5 = `rgba(244,67,54,0.3)` (red).
- Crowd pins:** Custom `Marker` for each venue:
 - Green circle pin for crowdSize 1-2, amber for 3, red for 4-5.
 - Venue name label below pin.
 - Tapping a pin shows a small callout with venue name, crowdSize, and `"View →"` link to `VenueScreen`.
- Deal pins:** For venues with active merchant ads, render a gold star `Marker`. Tapping shows the deal offer text with a `"Claim"` button.
- Filter chips** (floating row at top, above map): `All`, `Deals`, `Near me`. `All` = show everything. `Deals` = show only gold deal pins. `Near me` = restrict to 2km radius.
- Legend** (floating bottom-left): Small card showing green/amber/red dots with "Quiet / Moderate / Busy" labels.
- Data:** Fetch all venues with recent peeps from `GET /search/venues/nearby` and all active ads from `GET /merchant/feed`.

CONSTRAINTS

- Do not modify any files not listed above

- Do not change the navigation stack
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The map renders with the user's location centered and venue markers visible.
 2. Filter chips change which markers are shown.
-

Task 35 — Leaderboard screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 35.

PROBLEM

There is no leaderboard screen. Users need to see their rank and compare with others.

FILES TO EDIT

- `PeepIMobile/src/screens/LeaderboardScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `LeaderboardScreen.tsx`:

1. **"Your Stats" row at top:** Fixed header card (white on blue background) showing: Points (from `GET /leaderboard/rank`), Rank (#N), Pioneer Count. Three values in a horizontal row, each with a label below.
2. **Friends / Everyone toggle:** Two-segment control. `Everyone` fetches from `GET /leaderboard/everyone`. `Friends` fetches from `GET /leaderboard/friends`.
3. **Podium for top 3** (rendered above the scrollable list):
 - Gold (#1): Center position, tallest bar (height 80), crown icon, avatar, username.
 - Silver (#2): Left, medium bar (height 60).
 - Bronze (#3): Right, shorter bar (height 50). Each podium column has the user's avatar (circular, 44px) above the bar and their points below.
4. **Ranked list** (scrollable `FlatList` below podium): Rank number, avatar (40px circle), username, Pioneer badge (★ if `pioneerCount > 0`), points. Rows alternate white/light grey. Current user's row is highlighted with a blue left border.

5. **Loading state:** Show `ActivityIndicator` in center. **Error state:** Show retry button.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The podium renders correctly for the top 3 users with gold/silver/bronze bars.
 2. The current user's row is highlighted regardless of their position in the list.
-

Task 36 — Pioneers List screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 36.

PROBLEM

There is no Pioneers List screen showing who has pioneered the most venues.

FILES TO EDIT

- `PeepIMobile/src/screens/PioneersScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `PioneersScreen.tsx`:

1. **Header:** `"Pioneers"` title with ★ icon, blue background, white text.
2. **Data source:** `GET /leaderboard/everyone` — sort by `pioneerCount` descending (or add a dedicated endpoint — use the leaderboard endpoint and sort client-side for now).
3. **List** (`FlatList`): Each row:
 - Rank number (bold, 18pt, 40px wide).
 - Avatar: circular image (44px) using `profileImageUrl`, or grey placeholder.
 - Username (bold 16pt, black).
 - Pioneer badge: ★ icon + `"X venues"` count in gold.
 - `"Follow"` / `"Following"` button (right-aligned, small, outlined).

4. **Current user's row:** Highlighted with `background-color: #E3F2FD` background and blue left border (4px).
5. **Tapping a row:** Navigates to `UserProfile` screen with `userId`.
6. **Pull-to-refresh:** `onRefresh` prop on `FlatList`.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The list renders ranked users with pioneer counts correctly.
 2. The current user's row is highlighted.
-

Task 37 — Profile screen full rebuild

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 37.

PROBLEM

The current `ProfileScreen.tsx` is a basic placeholder. It must be rebuilt to match the full spec.

FILES TO EDIT

- `PeepIMobile/src/screens/ProfileScreen.tsx`

EXACT CHANGES REQUIRED

Rebuild `ProfileScreen.tsx`:

1. Hero section:

- Full-width hero photo (`height: 200`) from `user.profileImageUrl`. If null, use a solid blue rectangle.
- Gradient scrim (transparent to blue, bottom half).
- Circular avatar (80px) centered, with white border, overlapping the bottom of the hero.
- Name (white bold 22pt) below avatar.

- Pioneer badge: if `user.pioneerCount > 0`, show gold ★ pill badge with count.

2. Stats row (four tappable cards in a horizontal row):

- `Peeps` (count, tappable → `MyPeeps`)
- `Points` (count, tappable → leaderboard)
- `Followers` (count, tappable → `FollowList` with type=followers)
- `Following` (count, tappable → `FollowList` with type=following)

3. Menu rows (scrollable list below stats):

- My Peeps → `MyPeeps`
- Favorites → `Favorites`
- Points & Leaderboard → `Leaderboard`
- Likes → (placeholder for now)
- Places Pioneerred → `Pioneers`
- Groups → `Groups`
- Share Peep! → native share sheet with app download link
- Merchant / Advertise → `MerchantSignIn`
- Settings → `Settings`
- Log Out → calls `authService.signOut()` then navigates to `Login`

4. All data fetched from `GET /users/:uid` using current user's Firebase UID.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. Hero section, stats row, and all menu items render without layout errors.
2. Tapping `"Log Out"` signs out the user and navigates to the login screen.

Task 38 — Other user profile screen

CONTEXT

Peep! is a real-time crowd intelligence mobile app. This is Phase 3, Task 38.

PROBLEM

When viewing another user's profile (from leaderboard, pioneers, comments, etc.), the same profile screen is shown but it needs to display a Follow/Unfollow button instead of edit options.

FILES TO EDIT

- `PeepMobile/src/screens/UserProfileScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `UserProfileScreen.tsx` with the same hero section and stats row as `ProfileScreen` (Task 37), but with these differences:

1. **Route params:** `{ userId: string }`. Fetch user data from `GET /users/:userId`.
2. **Follow/Unfollow button:** Positioned top-right in the hero section. State: `"Follow"` (outlined white button) or `"Following"` (filled blue button). Tapping `"Follow"` calls `POST /users/:userId/follow`. Tapping `"Following"` calls `DELETE /users/:userId/follow`.
3. **No access to:** Settings, Log Out, Merchant/Advertise, Share Peep (these menu items are not shown).
4. **Recent Peeps section** (below stats row): `FlatList` of this user's peeps in reverse chronological order. Each row: venue name, crowd pill, timestamp.
5. **Check if viewing own profile:** If `userId === currentUser.uid`, redirect to `ProfileScreen` instead.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The Follow button calls `POST /users/:userId/follow` and toggles to `"Following"`.
 2. Viewing your own `userId` redirects to `ProfileScreen`.
-

Task 39 — Followers and Following list screens

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 39.

PROBLEM

There are no screens to view follower/following lists. A single reusable screen must handle both modes.

FILES TO EDIT

- `PeepIMobile/src/screens/FollowListScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `FollowListScreen.tsx`:

1. **Route params:** `{ userId: string, type: 'followers' | 'following' }`.
2. **Header:** Title = `"Followers"` or `"Following"` based on `type`. Back button.
3. **Search bar:** `TextInput` at top with search icon. Filters the displayed list client-side by username as the user types.
4. **Data fetch:** If `type === 'followers'`, fetch from `GET /users/:userId/followers`. If `type === 'following'`, fetch from `GET /users/:userId/following`.
5. **List** (`FlatList`): Each row:
 - Avatar: circular 44px image or grey placeholder.
 - Username (bold 16pt).
 - Pioneer badge: ★ shown if `user.pioneerCount > 0`.
 - Points count (grey 14pt).
 - Action button (right-aligned):
 - If viewing someone else's followers/following: show `"Follow"` / `"Following"` button.
 - If viewing own followers/following: show `"Remove"` for followers, `"Unfollow"` for following.
6. **Tapping a row:** Navigates to `UserProfile` with that user's `userId`.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. `type="followers"` fetches from the followers endpoint and `type="following"` from the following endpoint.
 2. The search bar filters the list in real time.
-

Task 40 — My Peeps history screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 40.

PROBLEM

There is no screen showing a user's own peep history.

FILES TO EDIT

- `PeepIMobile/src/screens/MyPeepsScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `MyPeepsScreen.tsx`:

1. **Header:** `"My Peeps"` title, blue background, white text. Back button.
2. **Data fetch:** `GET /peeps?userId=currentUser.uid` — returns peeps by the current user, reverse chronological.
3. **List** (`FlatList`): Each item:
 - **Venue thumbnail:** 80×80 image on the left (`borderRadius: 8`).
 - **Right side** (flex 1):
 - Venue name (bold 16pt).
 - Timestamp (grey 12pt, relative: "2h ago").
 - **Crowd pill:** Colored pill badge (`crowdSize` label).
 - **Vibe tags:** Horizontal row of small grey chips showing vibe values.
 - **Pioneer badge:** 🌟 `"Pioneer"` gold pill if `peep.isPioneer === true`.
 - **Like/Comment counts:** 🍷 X 💬 X row.
 - **Share action button:** `"Share"` tappable text on the far right that opens native share.
4. **Pull-to-refresh.**

5. **Empty state:** If no peeps, show "You haven't posted any Peeps yet." and a "Post your first Peep →" button navigating to `CreatePeep`.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The list renders with correct venue thumbnails and crowd pills.
 2. The Pioneer badge only appears on peeps where `isPioneer === true`.
-

Task 41 — Favorites list screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 41.

PROBLEM

There is no dedicated Favorites screen. Users need to view and manage their favorite venues.

FILES TO EDIT

- `PeepIMobile/src/screens/FavoritesScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `FavoritesScreen.tsx`:

1. **Header:** "Favorites" title, blue background, white text.
2. **Data fetch:** `GET /users/favorites` (from Task 24). Includes current crowd level and last peep time per venue.
3. **List** (`FlatList`): Each card (full-width, `height: 160`):
 - Full-width photo background from `venue.imageUrl`.
 - Gradient scrim.
 - Bottom-left: venue name (white bold 20pt), "Last peep: X ago" (white 12pt), distance from user.

- Top-right: crowd level circle (colored by crowdSize, same as rest of app).
 - Top-left: ❤️ filled heart icon — tapping it calls `DELETE /users/favorites/:venueId`, removes the card from the list with an animated fade-out.
4. **Crowd alerts toggle per venue:** Small bell icon next to the heart. Toggling saves preference to `AsyncStorage` keyed by `venueId`. (Push notification integration is a future task.)
 5. **Empty state:** `"No favorites yet. Tap the ♥ on any venue to save it."`.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. Favorites are fetched and displayed with current crowd data.
 2. Tapping the heart removes the venue from the list immediately (optimistic update).
-

Task 42 — Individual peep detail page

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 42.

PROBLEM

Tapping a peep anywhere in the app should open a full detail view. This screen does not exist.

FILES TO EDIT

- `PeepIMobile/src/screens/PeepDetailScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `PeepDetailScreen.tsx`:

1. **Route params:** `{ peepId: string }`. Fetch peep data from `GET /peeps/:peepId`.
2. **Hero photo:** Full-width image (`height: 240`) from `peep.photoUrl`. Gradient scrim bottom overlay.

3. **Venue name** (white bold 24pt) and **crowd pill** (colored circle with crowdSize) overlay on image.
4. **Timestamp** (white 12pt italic) on image.
5. **Reporter row:** Avatar (40px circle), username (bold), Pioneer badge (★ if applicable). Tapping navigates to `UserProfile`.
6. **Crowd data grid** (2-column): crowdSize label, M/F ratio display bar, A/K ratio display bar. Each cell has icon + label + value.
7. **Vibe tags:** Horizontal wrap of gold-outlined chips showing vibe values.
8. **Notes:** `peep.notes` text if present.
9. **Like / Comment / Share row:** Three equal buttons. Like button toggles via `POST/DELETE /peeps/:peepId/like`. Shows filled/outlined heart based on state.
10. **Likers avatars:** Horizontal row of up to 5 circular avatars of users who liked this peep. Tapping navigates to `Likers` screen.
11. **Comments list:** `FlatList` fetched from `GET /peeps/:peepId/comments`. Each row: avatar, username, comment text, timestamp.
12. **Comment input:** Fixed at the bottom (above keyboard): `TextInput` + `"Post"` button. Submits to `POST /peeps/:peepId/comments`.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. All crowd data fields (crowdSize, mfRatio, akRatio, ageRanges, vibe) render in the grid.
 2. The comment input posts a new comment and it appears in the comments list.
-

Task 43 — Likers screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 43.

PROBLEM

Tapping the likers avatars row on the peep detail screen should show a full list of who liked a

peep.

FILES TO EDIT

- `Peep1Mobile/src/screens/LikersScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `LikersScreen.tsx`:

1. **Route params:** `{ peepId: string }`.
2. **Header:** `"Liked by"` title.
3. **Data fetch:** `GET /peeps/:peepId/likes` — returns an array of `userIds`. For each `userId`, fetch the user doc. (Batch this: fetch all users in parallel with `Promise.all`.)
4. **List** (`FlatList`): Each row:
 - Avatar: circular 44px image or grey placeholder.
 - Username (bold 16pt).
 - Pioneer badge ★ if `user.pioneerCount > 0`.
 - `"Follow" / "Following"` button (right-aligned). Calls `POST/DELETE /users/:userId/follow`.
5. **Tapping a row:** Navigates to `UserProfile`.
6. **Loading state:** `ActivityIndicator` while fetching.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The screen fetches likers and displays them with username and avatar.
 2. Follow/Unfollow buttons update state correctly.
-

Task 44 — Groups screen

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 3, Task 44.

PROBLEM

There is no Groups screen. Users should be able to discover and join groups to share peeps privately with specific communities.

FILES TO EDIT

- `Peep1Mobile/src/screens/GroupsScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `GroupsScreen.tsx`:

1. **Header:** `"Groups"` title with `"+ Create"` button right-aligned. Create button shows an `Alert('Create group coming soon')` for now.
2. **Search bar:** Filters the discover groups list.
3. **"Your Groups" section** (horizontal `ScrollView` of cards): Only shown if user is in at least 1 group. Each card (`width: 140, height: 100`): group photo, group name, peep count today.
4. **"Discover Groups" section** (`FlatList`): Each card (full-width, `height: 80`):
 - Group photo (left, 60×60 rounded).
 - Group name (bold 16pt).
 - Member count and peeps today (grey 12pt).
 - `"Join"` / `"Joined"` button (right-aligned, small). `"Join"` calls `POST /groups/:groupId/join`. `"Joined"` calls `DELETE /groups/:groupId/leave`.
5. **Data:** `GET /groups` for discover list. Joined groups from user profile.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The Discover Groups list renders with Join/Joined buttons.
 2. Tapping `"Join"` calls the correct API and the button toggles to `"Joined"`.
-

Task 45 — Share screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 45.

PROBLEM

There is no Share screen for sharing peeps to followers, specific users, or groups.

FILES TO EDIT

- `PeepIMobile/src/screens/ShareScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `ShareScreen.tsx`:

1. **Route params:** `{ peepId: string }`.
2. **Header:** `"Share Peep"`.
3. **Peep preview card** at top: small card showing venue name, crowd pill, timestamp (the peep being shared).
4. **Share to Followers** section: Single large row button: `"Share to your followers"` with follower count subtitle. Tapping calls `POST /peeps/:peepId/share` (stub endpoint if not yet built — show success Alert for now).
5. **Share to a Person** section:
 - Search input: `placeholder="Search users"`. Debounced call to `GET /search/users?q=term`.
 - Results list: avatar + username + Follow state. Tapping a user selects them (highlighted in blue). `"Send"` button appears and calls the share endpoint.
6. **Share to a Group** section:
 - List of user's joined groups with checkboxes. `"Share to Group"` button calls `POST /groups/:groupId/share`.

7. **External Share** section: `"Share outside Peepl"` button triggers `Share.share()` from React Native with the text `"Check out what's happening at [venueName]: [deeplink]"`.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The peep preview card renders with the correct peep data.
 2. The external share button triggers the native OS share sheet.
-

Task 46 — Invite Friends screen

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 3, Task 46.

PROBLEM

There is no invite screen for users to invite their contacts to join Peepl.

FILES TO EDIT

- `PeeplMobile/src/screens/InviteScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `InviteScreen.tsx`:

1. **Header:** `"Invite Friends"`.
2. **Invite link section:**
 - Display the user's unique invite link: `"https://peepl.app/invite/[userId]"`.
 - `"Copy"` button: copies to clipboard using `Clipboard.setString()`.
 - `"Share"` button: triggers `Share.share()` with the link and text `"Join me on Peepl — know before you go! [link]"`.
3. **Contacts list:**

- `"Load contacts"` button — requests contacts permission, then loads contact list using `react-native-contacts`. (Show permission denial message gracefully if denied.)
- Renders contacts as a list: name, phone/email, `"Invite"` button per row.
- `"Invite"` button triggers SMS intent (iOS) or `Share.share()` (Android) with the invite message.
- After tapping `"Invite"`, the button changes to `"Invited ✓"` (grey, disabled). This state is persisted in `AsyncStorage` keyed by the contact's phone number.

4. **Pull-to-refresh** to reload contacts.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The invite link displays correctly with the current user's ID.
2. After tapping `"Invite"` for a contact, the button shows `"Invited ✓"` and persists on reload.

Task 47 — Menu / Navigation drawer

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 47.

PROBLEM

There is no full side-drawer menu. The navigation spec requires a hamburger-accessible drawer with all major sections.

FILES TO EDIT

- `PeepIMobile/src/screens/MenuScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `MenuScreen.tsx` as a slide-in drawer (rendered as a `Modal`) with slide animation from the

left):

1. **User card at top** (blue background, `height: 140`):
 - Avatar (60px circle).
 - Display name (white bold 18pt).
 - Pioneer status: gold "★ Pioneer" badge if `pioneerCount > 0`, otherwise "Start Peeping →".
 - Points count (white 14pt).
2. **Section: Discover** (grey section label):
 - Feed
 - Get Peeps
 - Map
 - Deals
3. **Section: Social:**
 - Share
 - Invite Friends
 - Leaderboard
 - Pioneers
 - Groups
4. **Section: Account:**
 - Profile
 - VIPeeps
 - Settings
5. **Each row:** Icon (left, 20pt), label (bold 15pt), right arrow (`>`).
6. **Hamburger icon** in the top-left of every main screen's header opens this menu via a shared state or context.
7. **Close behavior:** Tapping any item navigates to that screen and closes the menu. Tapping outside the drawer closes it.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change the navigation stack — that is Task 73
- Maintain existing blue/gold color scheme

- After changes, the app must still compile

VERIFICATION

1. The drawer slides in from the left and all menu items are tappable.
 2. Tapping a menu item navigates to the correct screen.
-

Task 48 — Search results screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 48.

PROBLEM

There is no dedicated search screen accessible from the app header.

FILES TO EDIT

- `PeepIMobile/src/screens/SearchScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `SearchScreen.tsx`:

1. **Search bar:** Auto-focused on mount. Debounced 300ms. Calls both `GET /search/venues?q=term` and `GET /search/users?q=term` in parallel for each query.
2. **Results: Venues section** (shown when results exist):
 - Header: `"Venues"`.
 - Each row: venue thumbnail (40px circle), venue name (bold), category (grey), distance from user, crowd pill.
 - Tapping → `VenueScreen`.
3. **Results: Users section** (shown when results exist):
 - Header: `"People"`.
 - Each row: avatar (40px circle), username (bold), Pioneer badge (★ if applicable), `"Follow"` / `"Following"` button.
 - Tapping → `UserProfile`.
4. **Recent Searches** (shown when search bar is empty):
 - `"Recent"` header.

- List of the last 5 searches stored in `AsyncStorage` key `"recentSearches"`.
- Tapping a recent search populates the search bar.
- `"Clear"` button removes all recent searches.

5. **On search submit:** Save the query to `AsyncStorage` recent searches (prepend, deduplicate, max 5 entries).

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. Typing a query triggers parallel venue and user searches.
2. Recent searches are saved and displayed when the search bar is empty.

Task 49 — Notifications inbox screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 49.

PROBLEM

There is no in-app notifications inbox. Users need to see a history of all their notifications.

FILES TO EDIT

- `PeepIMobile/src/screens/NotificationsScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `NotificationsScreen.tsx`:

1. **Header:** `"Notifications"` title. `"Mark all read"` button right-aligned.
2. **Data fetch:** `GET /notifications` (stub this endpoint in `server.js` to return from a `notifications` Firestore collection for the current user). Add the stub endpoint:

```
app.get('/notifications', authenticate, async (req, res) => { const snap =  
await admin.firestore().collection('notifications').where('userId', '==',
```

```
req.user.uid).orderBy('createdAt', 'desc').limit(50).get(); res.json({
  notifications: snap.docs.map(d => ({ id: d.id, ...d.data() })) }); });
```

3. **List** (`FlatList`): Each row:

- Unread rows: `#E3F2FD` background (light blue highlight).
- Icon (left, 32px circle): type-based icon.
- Message text (14pt): varies by type (see below).
- Timestamp (grey 11pt, relative).

4. **Notification types and messages:**

- `pioneer_buzz`: 🌟 "[Venue] is picking up – you discovered it first!"
- `crowd_spike`: 🔥 "[Venue] is getting crowded right now"
- `follower_peep`: 👁 "[Username] just peeped at [Venue]"
- `like`: ❤ "[Username] liked your Peep at [Venue]"
- `new_follower`: 👤 "[Username] started following you"
- `comment`: 💬 "[Username] commented on your Peep"
- `deal_alert`: 🎉 "New deal at [Venue]: [offerText]"

5. **Tapping a row:** Navigates to the relevant screen based on type (peep detail, venue, user profile, etc.) and marks the notification as read.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The screen fetches from the stub endpoint and renders notification rows with correct icons.
2. Unread notifications have a blue background highlight.

Task 50 — Settings screen

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 3, Task 50.

PROBLEM

There is no Settings screen matching the spec. It must be created as a new file.

FILES TO EDIT

- `Peep1Mobile/src/screens/SettingsScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `SettingsScreen.tsx`:

1. **Header:** `"Settings"` title, back button.
2. **Section: Notifications** (toggle rows — persist state in `AsyncStorage`):
 - Venue entry alerts (notify when I enter a venue I've favorited)
 - Favorited venue alerts (crowd spike notifications)
 - Deal alerts
 - Follower activity (when someone I follow posts a peep)
3. **Section: Privacy** (toggle rows — persist in user's Firestore doc via `PUT /users/profile`):
 - Location sharing (show my location to others)
 - Public profile (allow others to view my profile)
4. **Section: Account** (list rows with right arrows):
 - VIPeeps → `VIPeeps`
 - Help & Support → `Alert('Support: support@peep1.app')` for now
 - Log Out → confirm dialog → `authService.signOut()` → navigate to `Login`
5. **All toggles** use the blue active color (`#1565C0`).

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. All toggle rows render with correct initial state loaded from `AsyncStorage`.
 2. Log Out shows a confirmation dialog and navigates to Login on confirm.
-

Task 51 — Account Info screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 51.

PROBLEM

There is no Account Info screen for editing profile and managing account data.

FILES TO EDIT

- `PeepIMobile/src/screens/AccountInfoScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `AccountInfoScreen.tsx`:

1. **Header:** `"Account Info"` title, `"Save"` button right-aligned (disabled until a field changes).
2. **Profile photo:** Circular avatar (80px) with `"Change photo"` text below. Tapping opens image picker and updates `profileImageUrl` via `PUT /users/profile`.
3. **Editable fields** (white text inputs with grey labels above):
 - Full Name (maps to `firstName` + `lastName`)
 - Username
 - Bio (multiline, max 160 chars with character counter)
 - Email (non-editable, greyed out)
 - Phone (optional)
4. **Section: Account:**
 - Member since: `user.createdAt` formatted as `"Member since March 2025"`.
 - Plan: `"Free"` or `"VIPeeps"` with `"Upgrade →"` link.
 - Change Password: tapping sends a Firebase Auth password reset email and shows `"Reset email sent"` confirmation.
5. **Section: Data & Privacy:**
 - `"Download my data"` → `Alert('Data export coming soon')`.
 - `"Delete account"` → destructive confirm dialog → calls `DELETE /users/account` (stub endpoint for now).
6. **Save button** calls `PUT /users/profile` with changed fields only.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. All editable fields are populated from the current user's profile data on mount.
 2. The Save button calls `PUT /users/profile` only when a field has changed.
-

Task 52 — VIPeeps upsell screen

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 3, Task 52.

PROBLEM

There is no VIPeeps subscription screen. This is the primary monetization screen for consumer users.

FILES TO EDIT

- `Peep1Mobile/src/screens/VIPeepsScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `VIPeepsScreen.tsx`. This file handles both the upsell state and the active subscriber state (Task 53). Render based on `user.isVIP`.

Upsell state (`user.isVIP === false`):

1. **Header:** Blue gradient background, back button.
2. **Crown icon** 👑 (64pt, gold) centered.
3. **Title:** `"VIPeeps"` white bold 32pt.
4. **Price:** `"$4.99 / month"` gold 24pt.
5. **Feature list** (checkmark rows, white):
 - ✓ No ads — ever
 - ✓ Crowd history graphs for any venue
 - ✓ Deal alerts before everyone else

- ✓ Advanced crowd filters

6. **CTA button:** Full-width gold `"Start VIPeeps – $4.99/mo"`. For now: `Alert('Stripe integration coming in Task 68')`.
7. **Secondary link:** `"Stay on free plan"` in grey, small, below button. Navigates back.

CONSTRAINTS

- Do not modify any files not listed above
- Do not implement Stripe yet — that is Task 68
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The upsell state renders with all four feature rows visible.
 2. Tapping the CTA shows the `Alert` about Stripe integration.
-

Task 53 — VIPeeps active state screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 53.

PROBLEM

The `VIPeepsScreen.tsx` created in Task 52 needs a second state for active subscribers.

FILES TO EDIT

- `PeepIMobile/src/screens/VIPeepsScreen.tsx`

EXACT CHANGES REQUIRED

In `VIPeepsScreen.tsx`, add the active subscriber state rendered when `user.isVIP === true`:

1. **Header:** Same blue gradient.
2. **Active badge:** `"VIPeeps Active ✓"` — gold pill badge (gold background, black text), centered, large.
3. **Crown** 👑 (smaller, 48pt).
4. **Stats grid** (2×2 cards):

- Ads hidden: (count from user doc or `0` if not tracked)
- Deal alerts received: (count)
- Crowd graphs viewed: (count)
- Member since: formatted date

5. Billing info section:

- "Next billing date: [date]" (from `user.subscriptionRenewalDate`).
- "Payment method: ●●●● [last4]" (from `user.paymentLast4`).

6. "Cancel subscription" link — destructive confirm dialog → `Alert('Cancellation coming with full Stripe integration in Task 68')`.

Check `user.isVIP` at the top of `VIPeepsScreen.tsx` and render the appropriate state.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. Setting `user.isVIP = true` in the mock renders the active state, not the upsell.
2. Both states render without errors side by side in the simulator.

Task 54 — No Peeps Yet empty state

CONTEXT

Peopl is a real-time crowd intelligence mobile app. This is Phase 3, Task 54.

PROBLEM

When a venue has no peeps yet, `VenueScreen` shows nothing useful. A compelling empty state must be added to drive content creation.

FILES TO EDIT

- `PeoplMobile/src/screens/VenueScreen.tsx`

EXACT CHANGES REQUIRED

In `VenueScreen.tsx`, add a conditional block that renders when `venue.peepCount === 0` (or when the peeps array is empty after fetching):

1. Replace the peep list area with:
 - Faded 🗨 icon (60pt, grey, opacity 0.4), centered.
 - `"No peeps yet."` grey bold 18pt, centered.
 - `"Be the first to report what's happening at [venueName]."` grey 14pt, centered.
 - `"★ Be the Pioneer"` gold button (`#FFC107`, black text, full-width) — navigates to `CreatePeep` screen with `venueId` and `venueName` pre-filled in route params.
2. Make sure the hero image and venue info section above still render normally — only the peep feed area is replaced by the empty state.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change the hero or venue info sections of `VenueScreen`
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. Setting `venue.peepCount = 0` renders the empty state (not a blank screen).
 2. Tapping `"★ Be the Pioneer"` navigates to `CreatePeep` with the venue pre-filled.
-

Task 55 — Report a Peep screen

CONTEXT

Peopl is a real-time crowd intelligence mobile app. This is Phase 3, Task 55.

PROBLEM

Users have no way to report inaccurate or inappropriate peeps. A report screen must be built.

FILES TO EDIT

- `PeoplMobile/src/screens/ReportScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `ReportScreen.tsx`:

1. **Route params:** `{ peepId: string }`.
2. **Header:** `"Report Peep"` title, back button.
3. **Five tappable option cards** (full-width, white, `borderRadius: 12`, `marginBottom: 12`, with right arrow):
 - `"Inaccurate crowd info"` — subtitle: `"The crowd level or details don't match reality"`
 - `"Wrong venue"` — subtitle: `"This peep is tagged at the wrong location"`
 - `"Inappropriate content"` — subtitle: `"Photo or notes contain offensive content"`
 - `"Spam"` — subtitle: `"This is repeated or irrelevant content"`
 - `"Other"` — subtitle: `"Something else is wrong"`
4. **On tap:** Selected card gets a blue left border. A `"Submit Report"` button appears at the bottom (gold, `"Submit Report"`).
5. **On Submit:** Call `POST /peeps/:peepId/report` with body `{ reason: selectedReason }`. Add this stub route to `server.js`:

```
app.post('/peeps/:peepId/report',  
  authenticate, async (req, res) => { await  
    admin.firestore().collection('reports').add({ peepId: req.params.peepId,  
    reason: req.body.reason, reportedBy: req.user.uid, createdAt:  
    admin.firestore.FieldValue.serverTimestamp() }); res.json({ reported: true });  
  });
```
6. **After submit:** Show `"Thank you for your report. We'll review it shortly."` and auto-navigate back after 2 seconds.

CONSTRAINTS

- Do not modify any files not listed above except `server.js` to add the stub report endpoint
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. All five option cards render and are selectable (only one at a time).

- Submitting calls `POST /peeps/:peepId/report` and shows the confirmation message.
-

Task 56 — No connection state

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 3, Task 56.

PROBLEM

When the user's device goes offline, the app crashes or shows blank screens. A no-connection state must intercept the offline condition gracefully.

FILES TO EDIT

- `PeepIMobile/App.tsx`

EXACT CHANGES REQUIRED

- Install (if not already present) `@react-native-community/netinfo`.
- In `App.tsx`, import:

ts

```
import NetInfo from '@react-native-community/netinfo';
```

- Add state:

ts

```
const [isConnected, setIsConnected] = useState(true);  
const [lastUpdated, setLastUpdated] = useState<Date | null>(null);
```

- Add a `useEffect` that subscribes to `NetInfo.addEventListener(state => {
 setIsConnected(state.isConnected ?? true); if (state.isConnected)
 setLastUpdated(new Date()); })`.
- When `isConnected === false`, render the no-connection overlay **above** the navigation stack (not replacing it):

tsx


```

{!isConnected && (
  <View style={styles.offlineOverlay}>
    <Text style={styles.offlineIcon}>📶 </Text>
    <Text style={styles.offlineTitle}>No connection</Text>
    <Text style={styles.offlineBody}>Check your internet connection and try again</Text>
    {lastUpdated && (
      <Text style={styles.offlineTimestamp}>
        Last updated {Math.round((Date.now() - lastUpdated.getTime()) / 60000)} m
      </Text>
    )}
    <TouchableOpacity onPress={() => NetInfo.fetch().then(s => setIsConnected(s.isOnline))}>
      <Text style={styles.retryButton}>Try again</Text>
    </TouchableOpacity>
  </View>
)}

```

6. The `offlineOverlay` style: `position: 'absolute', top: 0, left: 0, right: 0, bottom: 0, backgroundColor: '#1565C0', justifyContent: 'center', alignItems: 'center', zIndex: 9999`.

CONSTRAINTS

- Do not modify any files not listed above
- Do not replace the navigation stack — use an overlay
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. Enabling airplane mode shows the offline overlay with the 📶 icon.
2. Re-enabling the network dismisses the overlay automatically.

PHASE 4 — Merchant Portal

Build the complete self-serve merchant advertising portal end-to-end. This is the primary revenue infrastructure.

Task 57 — Merchant data model

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 4, Task 57.

PROBLEM

The Firestore data model does not include the merchant-specific collections needed for the advertising portal. These collections must be defined and seeded.

FILES TO EDIT

- `server.js`
- `firestore.rules`

EXACT CHANGES REQUIRED

1. In `firestore.rules`, confirm the `merchant_accounts` and `merchant_ads` rules from Task 2 are present. If not, add them.
2. In `server.js`, add a documentation comment block defining the data schemas (this acts as schema documentation in code):

```
js
```

```

/*
MERCHANT DATA MODEL

Collection: merchant_accounts
Fields:
  - businessName: string
  - category: 'bar_pub' | 'restaurant' | 'coffee' | 'retail' | 'services' | 'ente
  - address: string
  - city: string
  - contactName: string
  - email: string
  - phone: string
  - paymentMethod: string (Stripe payment method ID)
  - merchantNumber: string (7-digit generated number, e.g. '#7547698')
  - createdAt: Timestamp
  - isActive: boolean
  - stripeCustomerId: string
  - ownerId: string (Firebase Auth UID)

Collection: merchant_ads
Fields:
  - merchantId: string (doc ID from merchant_accounts)
  - offerText: string (max 40 chars)
  - startTime: Timestamp
  - endTime: Timestamp
  - rateType: 'basic' | 'standard' | 'premium'
  - ratePerHour: number (10 | 25 | 50)
  - totalCost: number
  - status: 'pending' | 'live' | 'ended' | 'cancelled'
  - impressions: number
  - claims: number
  - venueRadius: number (km, default 1)
  - lat: number
  - lng: number
  - createdAt: Timestamp
*/

```

3. Add a utility function to `server.js` for generating merchant numbers:

```
js
```

```
function generateMerchantNumber() {  
  return '#' + Math.floor(1000000 + Math.random() * 9000000).toString();  
}
```

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `generateMerchantNumber()` exists in `server.js` and returns a string starting with `#` followed by 7 digits.
 2. The schema documentation comment is present and accurate.
-

Task 58 — Merchant sign in screen

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 4, Task 58.

PROBLEM

There is no merchant sign-in screen. Merchants use a separate authentication flow from consumers.

FILES TO EDIT

- `Peep1Mobile/src/screens/merchant/MerchantSignInScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `MerchantSignInScreen.tsx` in `Peep1Mobile/src/screens/merchant/`:

1. **Background:** Full blue gradient (`#1565C0` to `#0D47A1`).
2. **Title:** `"Peep1"` logo (white bold 52pt) + `"Merchant Portal"` subtitle (white 16pt italic), centered.
3. **Merchant # input:** White background `TextInput`, `placeholder="Merchant #"`, `keyboardType="numeric"`.

4. **Password input:** White background, `placeholder="Password"`, `secureTextEntry`.
5. **"Sign In to Portal" button** (gold `#FFC107`, black text, full-width). On press:
 - Calls `POST /merchant/signin` with `{ merchantNumber, password }`. Add this stub route to `server.js`:

```
js

app.post('/merchant/signin', async (req, res) => {
  const { merchantNumber, password } = req.body;
  const snap = await admin.firestore().collection('merchant_accounts')
    .where('merchantNumber', '==', merchantNumber).limit(1).get();
  if (snap.empty) return res.status(401).json({ error: 'Invalid credentials' });
  // TODO: verify password with Firebase Auth using email from doc
  const merchant = { id: snap.docs[0].id, ...snap.docs[0].data() };
  res.json({ merchant });
});
```

- On success: navigate to `MerchantPortal` with merchant data.
6. **"Set Up Merchant Account" button** (white outlined, white text) — navigates to `MerchantSetupStep1`.
 7. **"How to Advertise on Peep1" link** — navigates to `HowToAdvertise`.

CONSTRAINTS

- Do not modify any files not listed above except `server.js` for the stub signin route
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The screen renders with all three interactive elements.
 2. Tapping `"Set Up Merchant Account"` navigates to `MerchantSetupStep1`.
-

Task 59 — Merchant setup Step 1 — business info

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 4, Task 59.

PROBLEM

There is no merchant account setup flow. Step 1 collects business information.

FILES TO EDIT

- `Peep1Mobile/src/screens/merchant/MerchantSetupStep1Screen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `MerchantSetupStep1Screen.tsx`:

1. **Progress bar:** Thin blue bar at top. Step 1 of 3 = 33% filled. `"Step 1 of 3"` text below.
2. **Title:** `"Your Business"` white bold 24pt.
3. **Form fields** (white inputs, `borderRadius: 12`):
 - Business Name (`placeholder="Business name"`), required)
 - Address (`placeholder="Street address"`), required)
 - City (`placeholder="City"`), required)
4. **Category selection grid** (tappable cards in 2-column grid):
 - Bar / Pub
 - Restaurant
 - Coffee Shop
 - Retail
 - Services
 - Entertainment
 - Other Each card: emoji + label. Selected card: blue border + blue background tint.
5. **Validation:** Business name, address, city, and category are all required. Show inline red error text if blank on Next press.
6. `"Next →"` **button** (gold, full-width) — saves data to a local state object (using React context or passed as params) and navigates to `MerchantSetupStep2` with the business data.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. All 7 category cards render in a 2-column grid.
 2. Tapping `"Next"` without filling required fields shows validation errors.
-

Task 60 — Merchant setup Step 2 — billing

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 4, Task 60.

PROBLEM

Step 2 of the merchant setup collects contact and billing information.

FILES TO EDIT

- `Peep1Mobile/src/screens/merchant/MerchantSetupStep2Screen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `MerchantSetupStep2Screen.tsx`:

1. **Progress bar:** Step 2 of 3 = 66% filled.
2. **Title:** `"Contact & Billing"` white bold 24pt.
3. **Contact fields** (white inputs):
 - Contact Name (required)
 - Email (required, `keyboardType="email-address"`)
 - Phone (required, `keyboardType="phone-pad"`)
4. **Payment section:**
 - Title: `"Payment Method"` bold.
 - For now: a placeholder card-entry UI (grey card-shaped container with `" Stripe card entry will appear here"` text — actual Stripe integration is Task 68). Add a

note in comments: `// TODO: Replace with @stripe/stripe-react-native`
`CardField` in Task 68).

5. **Rate explanation text** (white, 13pt, centered):

"You are only charged for active ad time – no minimums, no commitments.
Basic: \$10/hr • Standard: \$25/hr • Premium: \$50/hr"

6. **"Next ➔" button** (gold) — validates all fields, then calls `POST /merchant/setup` to create the merchant account:

```
js
// Add to server.js
app.post('/merchant/setup', authenticate, async (req, res) => {
  const { businessName, category, address, city, contactName, email, phone } = req.body
  const merchantNumber = generateMerchantNumber()
  const docRef = await admin.firestore().collection('merchant_accounts').add({
    businessName, category, address, city, contactName, email, phone,
    merchantNumber, ownerId: req.user.uid,
    isActive: true, createdAt: admin.firestore.FieldValue.serverTimestamp(),
  });
  res.json({ merchantId: docRef.id, merchantNumber });
});
```

On success, navigates to `MerchantAccountNumber` with `{ merchantId, merchantNumber }`.

CONSTRAINTS

- Do not modify any files not listed above except `server.js` for the setup endpoint
- Do not implement Stripe yet
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. All contact fields are required and validated before `"Next"` proceeds.
2. Tapping `"Next"` with valid data calls `POST /merchant/setup` and navigates to `MerchantAccountNumber`.

Task 61 — Merchant account number screen

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 4, Task 61.

PROBLEM

After completing merchant setup, the merchant receives their unique account number and must be able to note it.

FILES TO EDIT

- `PeepIMobile/src/screens/merchant/MerchantAccountNumberScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `MerchantAccountNumberScreen.tsx`:

1. **Route params:** `{ merchantId: string, merchantNumber: string }`.
2. **Background:** Blue gradient.
3. **Large** 🎉 emoji (72pt), centered.
4. **Title:** `"You're live on PeepI!"` white bold 28pt.
5. **Merchant number display:** The `merchantNumber` string (e.g., `#7547698`) in gold bold 48pt, centered. Wrapped in a white-bordered rounded box for emphasis.
6. **Instruction text:** `"This is your Merchant #. You'll need it to sign in to your portal."` white 14pt, centered.
7. `"Go to Merchant Portal ➔"` button (gold, full-width) — navigates to `MerchantPortal` and clears the setup stack so back navigation is not possible.
8. `"Save to notes app"` link (white, small, secondary) — calls `Share.share({ message: 'My PeepI Merchant # is ' + merchantNumber })`.
9. **Keep screen awake** while displayed: call `KeepAwake.activate()` on mount, `KeepAwake.deactivate()` on unmount.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The merchant number renders in gold 48pt with the correct value from route params.
 2. `KeepAwake.activate()` is called on mount.
-

Task 62 — Self-serve ad submission portal (3-step flow)

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 4, Task 62.

PROBLEM

Merchants need a self-serve interface to create and submit ads. This is the core revenue feature of the portal.

FILES TO EDIT

- `Peep1Mobile/src/screens/merchant/MerchantPortalScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `MerchantPortalScreen.tsx` with a 3-step form rendered in a single screen using step state:

1. **Merchant header** (top card, always visible): Business name (bold 18pt), merchant number (grey 13pt), `"● Active"` green status dot.
2. **Step 1 — Your Offer:**
 - Label: `"Write your offer"` bold.
 - `TextInput` multiline, max 40 chars. Character counter below (`"X / 40"`).
 - Example hint: `"e.g. '50% off all cocktails tonight'"`.
 - `"Next →"` validates non-empty.
3. **Step 2 — Time & Rate:**
 - Label: `"Start time"` — tappable field that opens `DateTimePicker` (or a `Modal` with time wheels). Shows selected time in `"Today at 8:00 PM"` format.
 - Label: `"End time"` — same. Must be after start time.
 - **Rate selection** (3 tappable cards):
 - `"Basic — $10/hr"` with `"Standard reach"` subtitle.
 - `"Standard — $25/hr"` with `"2× reach + highlighted"` subtitle.

- `"Premium – $50/hr"` with `"Maximum reach + deal alerts"` subtitle.
- Selected rate card: blue border.
- `"Next →"` validates both times and rate selected.

4. Step 3 — Review:

- `"Review your ad"` title.
- Summary card: offer text, start/end time, rate, duration (calculated hours), `"Total: $XX.XX"` (duration × rate).
- `"Submit Ad"` button (gold, full-width) — calls `POST /merchant/ads` with all data. On success: show `Alert('Your ad is live!')` and reset to Step 1.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. All three steps render and the step indicator updates.
2. Step 3 shows the correct calculated total cost based on duration and rate.

Task 63 — Merchant activity dashboard

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 4, Task 63.

PROBLEM

Merchants need a dashboard to see how their ads are performing. This screen does not exist.

FILES TO EDIT

- `PeepIMobile/src/screens/merchant/MerchantActivityScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `MerchantActivityScreen.tsx`:

1. **Route params:** `{ merchantId: string }`.
2. **Data fetch:** `GET /merchant/ads?merchantId=merchantId` (add this endpoint to `server.js`: returns all ads for the merchant from `merchant_ads` collection ordered by `createdAt` descending).
3. **Four stat boxes** (2×2 grid, white cards):
 - Total Impressions (sum across all ads)
 - Deal Claims (sum of `claims`)
 - Claim Rate % (`((claims / impressions * 100).toFixed(1) + '%')`)
 - Total Spent (sum of `totalCost` for ended/live ads)
4. **Bar chart** of impressions by hour (last 24 hours). Use `react-native-svg` to render a simple bar chart: 24 bars, x-axis labels every 4 hours, y-axis from 0 to max impressions. Color: blue (`#1565C0`).
5. **Active campaigns list** (status === 'live'): Each row: offer text (bold), start-end time, impressions count, claims count, spend. Collapsible via tap.
6. **Ended campaigns list** (status === 'ended'): Same format, below active. Grey text. Collapsible section header.

CONSTRAINTS

- Do not modify any files not listed above except `server.js` for the merchant ads endpoint
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. The four stat boxes display calculated totals correctly.
2. The bar chart renders with 24 bars without crashing.

Task 64 — Merchant account info / billing screen

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 4, Task 64.

PROBLEM

Merchants need a screen to view their account details, billing info, and rate reference.

FILES TO EDIT

- `Peep1Mobile/src/screens/merchant/MerchantAccountInfoScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `MerchantAccountInfoScreen.tsx`:

1. **Header:** `"Account"` title, blue background, back button.
2. **Section: Business** (read-only display rows):
 - Business Name
 - Merchant #
 - Category
 - Address
3. **Section: Billing** (read-only display rows):
 - Payment Method: `"●●●● [last4]"` (from `merchant.paymentLast4`, show `"Not set"` if null — Stripe integration in Task 68 will populate this).
 - Billing Email: `merchant.email`.
 - Total Spent: sum of all ad costs.
 - `"View invoices"` link — `Alert('Invoice download coming soon')`.
4. **Section: Rates Reference** (informational card):
 - Basic: `$10/hr – Standard reach`
 - Standard: `$25/hr – 2× reach + highlighted`
 - Premium: `$50/hr – Maximum reach + deal alerts`
5. **Section: Support:**
 - `"Help & Support"` → `Alert('support@peep1.app')`.
6. **Danger zone:**
 - `"Close merchant account"` in red — destructive confirm dialog → `Alert('Account closure coming soon')`.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing blue/gold color scheme
- After changes, the app must still compile

VERIFICATION

1. All four sections render with data from the merchant object passed via route params.
 2. The Rates Reference section shows all three tiers.
-

Task 65 — How to Advertise screen

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 4, Task 65.

PROBLEM

There is no educational screen explaining how merchant advertising works on Peepl. This is shown from the Merchant Sign In screen.

FILES TO EDIT

- `PeeplMobile/src/screens/merchant/HowToAdvertiseScreen.tsx` — CREATE NEW FILE

EXACT CHANGES REQUIRED

Create `HowToAdvertiseScreen.tsx`:

1. **Background:** White (not blue — this is a content/marketing screen).
2. **Header:** `"How Advertising Works"` title, blue background, back button, white text.
3. **Five numbered steps** (rendered as cards, each with a number badge, title, and description):
 1. **Create your offer** — `"Write a short offer (up to 40 characters). Keep it punchy - 'Half-price drinks until 10pm' works great."` Icon: 
 2. **Pick your time slot** — `"Choose when your ad runs. Start any time, end any time. You control the schedule."` Icon: 
 3. **Choose your rate** — `"Basic ($10/hr), Standard ($25/hr), or Premium ($50/hr). Higher rate = more reach and visibility."` Icon: 
 4. **Go live instantly** — `"Your ad goes live at your chosen start time. No approval process, no waiting."` Icon: 
 5. **See your results** — `"Track impressions, deal claims, and claim rate in your activity dashboard."` Icon: 

4. `"Start Advertising →"` button (gold, full-width, at bottom) — navigates to `MerchantSignIn`.

CONSTRAINTS

- Do not modify any files not listed above
- After changes, the app must still compile

VERIFICATION

1. All five numbered steps render with their icons, titles, and descriptions.
 2. The CTA button navigates to `MerchantSignIn`.
-

Task 66 — Backend: merchant ad serving logic

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 4, Task 66.

PROBLEM

There is no backend logic to serve merchant ads, handle the pending→live→ended lifecycle, record impressions, or handle claims.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

Add the following to `server.js`:

1. **Create ad endpoint:**

```
js
```

```

app.post('/merchant/ads', authenticate, async (req, res) => {
  const { merchantId, offerText, startTime, endTime, rateType } = req.body;
  if (!offerText || offerText.length > 40) return res.status(400).json({ error: 'Offer text is too long' });
  const rates = { basic: 10, standard: 25, premium: 50 };
  const ratePerHour = rates[rateType];
  if (!ratePerHour) return res.status(400).json({ error: 'Invalid rate type' });
  const start = new Date(startTime);
  const end = new Date(endTime);
  const hours = (end - start) / 3600000;
  if (hours <= 0) return res.status(400).json({ error: 'End time must be after start time' });
  const totalCost = parseFloat((hours * ratePerHour).toFixed(2));
  const merchantDoc = await admin.firestore().collection('merchant_accounts').doc(merchantId).get();
  if (!merchantDoc.exists) return res.status(404).json({ error: 'Merchant not found' });
  const adRef = await admin.firestore().collection('merchant_ads').add({
    merchantId, offerText, startTime: admin.firestore.Timestamp.fromDate(start),
    endTime: admin.firestore.Timestamp.fromDate(end), rateType, ratePerHour,
    totalCost, status: start <= new Date() ? 'live' : 'pending',
    impressions: 0, claims: 0, createdAt: admin.firestore.FieldValue.serverTimestamp(),
    lat: merchantDoc.data().lat || null, lng: merchantDoc.data().lng || null,
  });
  res.json({ adId: adRef.id, totalCost });
});

```

2. Active ads near a location (for feed injection):

```

js

app.get('/merchant/feed', authenticate, async (req, res) => {
  const { lat, lng } = req.query;
  const snap = await admin.firestore().collection('merchant_ads')
    .where('status', '==', 'live')
    .limit(20)
    .get();
  // Filter by distance client-side for now (Task 67 will add geohash)
  const ads = snap.docs.map(d => ({ id: d.id, ...d.data() }));
  // Increment impressions for each returned ad
  const batch = admin.firestore().batch();
  snap.docs.forEach(d => batch.update(d.ref, { impressions: admin.firestore.FieldValue.increment(1) }));
  await batch.commit();
  res.json({ ads });
});

```


3. Claim an ad:

```
js

app.post('/merchant/ads/:adId/claim', authenticate, async (req, res) => {
  const adRef = admin.firestore().collection('merchant_ads').doc(req.params.adId)
  await adRef.update({ claims: admin.firestore.FieldValue.increment(1) });
  res.json({ claimed: true });
});
```

4. Cron scheduler to flip ad statuses:

```
js

const cron = require('node-cron');
cron.schedule('* * * * *', async () => {
  const now = admin.firestore.Timestamp.now();
  // Flip pending → live
  const pendingSnap = await admin.firestore().collection('merchant_ads')
    .where('status', '==', 'pending').where('startTime', '<=', now).get();
  // Flip live → ended
  const liveSnap = await admin.firestore().collection('merchant_ads')
    .where('status', '==', 'live').where('endTime', '<=', now).get();
  const batch = admin.firestore().batch();
  pendingSnap.docs.forEach(d => batch.update(d.ref, { status: 'live' }));
  liveSnap.docs.forEach(d => batch.update(d.ref, { status: 'ended' }));
  await batch.commit();
});
```

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `POST /merchant/ads` creates a document with correct `totalCost` calculation.
 2. The cron job runs every minute and flips ad statuses correctly.
-

PHASE 5 — Infrastructure, Polish & Launch Prep

Everything needed to get to app store submission.

Task 67 — GeoHash implementation for venue search

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 5, Task 67.

PROBLEM

Venue search and geofencing still perform full Firestore collection scans in some places. All geospatial queries must use GeoHash-based range queries for scalability.

FILES TO EDIT

- `server.js`
- `GeofencingService.js`
- `scripts/init-database.js`

EXACT CHANGES REQUIRED

1. Confirm `geofire-common` is in `package.json` dependencies (added in Task 6). If not, add it.
2. In `server.js`, ensure all geospatial venue queries use the GeoHash pattern from Task 6. Audit all routes that query the `venues` collection for proximity and replace any remaining collection scans.
3. In `GeofencingService.js`, replace any remaining full `venues` collection queries with GeoHash-based queries using `geofire.geohashQueryBounds()`. Import `geofire-common` at the top if not already done.
4. In `scripts/init-database.js`, for each venue object being seeded, add a `geohash` field:

js

```
const geofire = require('geofire-common');
// For each venue:
venue.geohash = geofire.geohashForPoint([venue.lat, venue.lng]);
venue.nameLower = venue.name.toLowerCase();
venue.cityLower = venue.city.toLowerCase();
```

5. Create a Firestore index (document the required index in a comment in `server.js`):

```
js

// Required Firestore index: collection=venues, fields=[geohash ASC]
// Create via Firebase Console or firestore.indexes.json
```

CONSTRAINTS

- Do not modify any files not listed above
- Do not change the logic of geofencing — only the query mechanism
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `GeofencingService.js` contains no full collection `.get()` calls on the `venues` collection — all proximity queries use `startAt/endAt` on `geohash`.
 2. `init-database.js` generates `geohash`, `nameLower`, and `cityLower` fields for every venue.
-

Task 68 — Stripe integration for billing

CONTEXT

Peep! is a real-time crowd intelligence mobile app. This is Phase 5, Task 68.

PROBLEM

Merchant ad billing and VIPeeps subscriptions are stubbed out. Real Stripe integration must be built.

FILES TO EDIT

- `server.js`
- `Peep!Mobile/src/screens/merchant/MerchantSetupStep2Screen.tsx`

EXACT CHANGES REQUIRED

1. Add `stripe` to `package.json` dependencies. Add `STRIPE_SECRET_KEY` to environment variables. Initialize Stripe in `server.js`:

js

```
const Stripe = require('stripe');  
const stripe = Stripe(process.env.STRIPE_SECRET_KEY);
```

2. Add a route to create a Stripe customer during merchant setup:

js

```
app.post('/merchant/create-customer', authenticate, async (req, res) => {  
  const { email, businessName } = req.body;  
  const customer = await stripe.customers.create({ email, name: businessName });  
  await admin.firestore().collection('merchant_accounts').doc(req.body.merchantId)  
    .update({ stripeCustomerId: customer.id });  
  res.json({ customerId: customer.id });  
});
```

3. Add a route to handle VIPeeps subscription creation:

js

```
app.post('/vipeeps/subscribe', authenticate, async (req, res) => {
  const { paymentMethodId } = req.body;
  const userDoc = await admin.firestore().collection('users').doc(req.user.uid).get();
  let customerId = userDoc.data()?.stripeCustomerId;
  if (!customerId) {
    const customer = await stripe.customers.create({ email: userDoc.data().email });
    customerId = customer.id;
    await admin.firestore().collection('users').doc(req.user.uid).update({ stripeCustomerId: customerId });
  }
  await stripe.paymentMethods.attach(paymentMethodId, { customer: customerId });
  const subscription = await stripe.subscriptions.create({
    customer: customerId,
    items: [{ price: process.env.STRIPE_VIPEEPS_PRICE_ID }],
    default_payment_method: paymentMethodId,
  });
  await admin.firestore().collection('users').doc(req.user.uid).update({
    isVIP: true,
    subscriptionId: subscription.id,
    subscriptionRenewalDate: new Date(subscription.current_period_end * 1000).toISOString(),
  });
  res.json({ subscriptionId: subscription.id });
});
```

4. Add Stripe webhook handler:

js

```

app.post('/webhooks/stripe', express.raw({ type: 'application/json' }), async (req, res) => {
  const sig = req.headers['stripe-signature'];
  let event;
  try {
    event = stripe.webhooks.constructEvent(req.body, sig, process.env.STRIPE_WEBHOOK_SECRET);
  } catch (err) {
    return res.status(400).send(`Webhook error: ${err.message}`);
  }
  if (event.type === 'invoice.payment_failed') {
    // TODO: notify user and flag account
  }
  if (event.type === 'customer.subscription.deleted') {
    const subId = event.data.object.id;
    const snap = await admin.firestore().collection('users').where('subscriptionId', '==', subId).get();
    snap.docs.forEach(d => d.ref.update({ isVIP: false }));
  }
  res.json({ received: true });
});

```

5. In `MerchantSetupStep2Screen.tsx`, replace the Stripe placeholder with the actual `@stripe/stripe-react-native` `CardField` component:

tsx

```

import { CardField, useConfirmPayment } from '@stripe/stripe-react-native';
// Replace the placeholder with:
<CardField
  postalCodeEnabled={false}
  style={{ height: 50, marginVertical: 8 }}
  onCardChange={(cardDetails) => setCardDetails(cardDetails)}
/>

```

CONSTRAINTS

- Do not modify any files not listed above
- Do not use real Stripe keys in code — use environment variables
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `server.js` has a working `/vipeeps/subscribe` endpoint and Stripe webhook handler.
 2. `MerchantSetupStep2Screen.tsx` uses `CardField` from `@stripe/stripe-react-native` instead of the placeholder.
-

Task 69 — Social graph backend

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 5, Task 69.

PROBLEM

The follow/unfollow, followers, and following endpoints do not exist. The social graph backend must be built to support the profile and leaderboard screens.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

Add the following routes to `server.js` before `listen()`:

1. Follow a user:

```
js

app.post('/users/:userId/follow', authenticate, async (req, res) => {
  const { userId } = req.params;
  if (userId === req.user.uid) return res.status(400).json({ error: 'Cannot follow self' });
  const db = admin.firestore();
  await db.collection('follows').doc(`${req.user.uid}_${userId}`).set({
    followerId: req.user.uid,
    followingId: userId,
    createdAt: admin.firestore.FieldValue.serverTimestamp(),
  });
  await db.collection('users').doc(userId).update({ followersCount: admin.firestore.FieldValue.increment(1) });
  await db.collection('users').doc(req.user.uid).update({ followingCount: admin.firestore.FieldValue.increment(1) });
  res.json({ following: true });
});
```

2. Unfollow:

```
js

app.delete('/users/:userId/follow', authenticate, async (req, res) => {
  const { userId } = req.params;
  await admin.firestore().collection('follows').doc(`${req.user.uid}_${userId}`).delete();
  await admin.firestore().collection('users').doc(userId).update({ followersCount: req.user.followersCount - 1 });
  await admin.firestore().collection('users').doc(req.user.uid).update({ following: false });
  res.json({ following: false });
});
```

3. Get followers:

```
js

app.get('/users/:userId/followers', authenticate, async (req, res) => {
  const snap = await admin.firestore().collection('follows')
    .where('followingId', '==', req.params.userId)
    .orderBy('createdAt', 'desc').limit(50).get();
  const users = await Promise.all(snap.docs.map(async d => {
    const userDoc = await admin.firestore().collection('users').doc(d.data().followerId).get();
    return { userId: d.data().followerId, ...userDoc.data() };
  }));
  res.json({ followers: users });
});
```

4. Get following:

```
js
```



```

app.get('/users/:userId/following', authenticate, async (req, res) => {
  const snap = await admin.firestore().collection('follows')
    .where('followerId', '==', req.params.userId)
    .orderBy('createdAt', 'desc').limit(50).get();
  const users = await Promise.all(snap.docs.map(async d => {
    const userDoc = await admin.firestore().collection('users').doc(d.data().followingId).get();
    return { userId: d.data().followingId, ...userDoc.data() };
  }));
  res.json({ following: users });
});

```

5. Following feed:

```

js
app.get('/feed/following', authenticate, async (req, res) => {
  const followingSnap = await admin.firestore().collection('follows')
    .where('followerId', '==', req.user.uid).get();
  const followingIds = followingSnap.docs.map(d => d.data().followingId);
  if (followingIds.length === 0) return res.json({ peeps: [] });
  const chunks = [];
  for (let i = 0; i < followingIds.length; i += 10) chunks.push(followingIds.slice(i, i + 10));
  const peeps = (await Promise.all(
    chunks.map(chunk => admin.firestore().collection('peeps').where('userId', 'in', chunk).get())
  )).flatMap(s => s.docs.map(d => ({ id: d.id, ...d.data() })));
  peeps.sort((a, b) => b.createdAt?.seconds - a.createdAt?.seconds);
  res.json({ peeps: peeps.slice(0, 50) });
});

```

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `POST /users/:userId/follow` creates a document in the `follows` collection and increments both users' counts.
2. `GET /feed/following` returns peeps only from followed users.

Task 70 — Groups backend

CONTEXT

Peopl is a real-time crowd intelligence mobile app. This is Phase 5, Task 70.

PROBLEM

The Groups feature has a frontend (Task 44) but no backend. The group management endpoints must be built.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

Add the following routes to `server.js` before `listen()`:

1. Create a group:

```
js

app.post('/groups', authenticate, async (req, res) => {
  const { name, description, isPublic = true } = req.body;
  if (!name) return res.status(400).json({ error: 'Group name required' });
  const groupRef = await admin.firestore().collection('groups').add({
    name, description: description || '', isPublic, creatorId: req.user.uid,
    memberCount: 1, peepsToday: 0, createdAt: admin.firestore.FieldValue.serverTimestamp()
  });
  await groupRef.collection('members').doc(req.user.uid).set({ joinedAt: admin.firestore.FieldValue.serverTimestamp() });
  res.json({ groupId: groupRef.id });
});
```

2. Discover groups:

```
js
```

```

app.get('/groups', authenticate, async (req, res) => {
  const snap = await admin.firestore().collection('groups')
    .where('isPublic', '==', true).orderBy('memberCount', 'desc').limit(20).get()
  res.json({ groups: snap.docs.map(d => ({ id: d.id, ...d.data() })) });
});

```

3. Join a group:

```

js

app.post('/groups/:groupId/join', authenticate, async (req, res) => {
  const { groupId } = req.params;
  const groupRef = admin.firestore().collection('groups').doc(groupId);
  await groupRef.collection('members').doc(req.user.uid).set({ joinedAt: admin.fi
  await groupRef.update({ memberCount: admin.firestore.FieldValue.increment(1) })
  res.json({ joined: true });
});

```

4. Leave a group:

```

js

app.delete('/groups/:groupId/leave', authenticate, async (req, res) => {
  const { groupId } = req.params;
  const groupRef = admin.firestore().collection('groups').doc(groupId);
  await groupRef.collection('members').doc(req.user.uid).delete();
  await groupRef.update({ memberCount: admin.firestore.FieldValue.increment(-1) })
  res.json({ joined: false });
});

```

5. Group feed:

```

js

```

```
app.get('/groups/:groupId/peeps', authenticate, async (req, res) => {
  const { groupId } = req.params;
  const snap = await admin.firestore().collection('peeps')
    .where('sharedToGroups', 'array-contains', groupId)
    .orderBy('createdAt', 'desc').limit(50).get();
  res.json({ peeps: snap.docs.map(d => ({ id: d.id, ...d.data() })) });
});
```

6. Share a peep to a group:

```
js
app.post('/groups/:groupId/share', authenticate, async (req, res) => {
  const { groupId } = req.params;
  const { peepId } = req.body;
  await admin.firestore().collection('peeps').doc(peepId).update({
    sharedToGroups: admin.firestore.FieldValue.arrayUnion(groupId),
  });
  res.json({ shared: true });
});
```

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `POST /groups` creates a group with the creator added as a member with role `"admin"`.
2. `GET /groups/:groupId/peeps` returns only peeps that have `groupId` in their `sharedToGroups` array.

Task 71 — Crowd history sparkline data

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 5, Task 71.

PROBLEM

The venue page (Task 20) uses a sparkline chart showing 12-hour crowd history, but the backend endpoint that powers it does not exist.

FILES TO EDIT

- `server.js`

EXACT CHANGES REQUIRED

Add the following route to `server.js` before `listen()`:

```
js
```

```
app.get('/venues/:venueId/crowd-history', authenticate, async (req, res) => {
  const { venueId } = req.params;
  const { hours = 12 } = req.query;
  const hoursInt = Math.min(Math.max(parseInt(hours), 1), 24);
  const since = new Date(Date.now() - hoursInt * 60 * 60 * 1000);

  const snap = await admin.firestore().collection('peeps')
    .where('venueId', '==', venueId)
    .where('createdAt', '>=', admin.firestore.Timestamp.fromDate(since))
    .orderBy('createdAt', 'asc')
    .get();

  // Bucket by hour
  const buckets = {};
  for (let i = 0; i < hoursInt; i++) {
    const bucketTime = new Date(since.getTime() + i * 60 * 60 * 1000);
    const key = bucketTime.toISOString().slice(0, 13); // 'YYYY-MM-DDTHH'
    buckets[key] = { hour: key, peepCount: 0, totalCrowdSize: 0, avgCrowdSize: 0 };
  }

  snap.docs.forEach(doc => {
    const data = doc.data();
    const peepTime = data.createdAt.toDate();
    const key = peepTime.toISOString().slice(0, 13);
    if (buckets[key]) {
      buckets[key].peepCount++;
      buckets[key].totalCrowdSize += data.crowdSize || 0;
      buckets[key].avgCrowdSize = buckets[key].totalCrowdSize / buckets[key].peepCount;
    }
  });

  const dataPoints = Object.values(buckets).map(b => ({
    hour: b.hour,
    peepCount: b.peepCount,
    avgCrowdSize: parseFloat(b.avgCrowdSize.toFixed(1)),
  }));

  res.json({ venueId, hours: hoursInt, dataPoints });
});
```

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `GET /venues/:venueId/crowd-history` returns exactly `hours` data points (default 12) even if some buckets have 0 peeps.
 2. Each data point contains `hour`, `peepCount`, and `avgCrowdSize`.
-

Task 72 — VIPeeps feature flagging

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 5, Task 72.

PROBLEM

VIPeeps subscribers should have a different experience (no ads, crowd history graphs, deal alerts, advanced filters) but there is no system to enforce these feature flags.

FILES TO EDIT

- `server.js`
- `PeeplMobile/App.tsx`

EXACT CHANGES REQUIRED

1. In `server.js`, create a `requireVIP` middleware:

```
js
```

```
const requireVIP = async (req, res, next) => {
  const userDoc = await admin.firestore().collection('users').doc(req.user.uid).get()
  if (!userDoc.data()?.isVIP) {
    return res.status(403).json({ error: 'VIPeeps subscription required' });
  }
  next();
};
```

2. Apply `requireVIP` to the crowd history endpoint (Task 71) by chaining it after `authenticate`:

```
js
app.get('/venues/:venueId/crowd-history', authenticate, requireVIP, async (req, res) => {
```

3. In `GET /merchant/feed`, if the requesting user is VIP, add deal alerts before the standard ads. Add to the response: `{ ads, isVIP: userDoc.data()?.isVIP || false }`. The mobile client can use `isVIP` to render enhanced deal alerts.
4. In `Peep1Mobile/App.tsx`, create a React Context for VIP status:

```
ts
import React, { createContext, useContext, useState, useEffect } from 'react';

type VIPContextType = { isVIP: boolean };
export const VIPContext = createContext<VIPContextType>({ isVIP: false });

export function VIPProvider({ children }: { children: React.ReactNode }) {
  const [isVIP, setIsVIP] = useState(false);
  useEffect(() => {
    // Fetch from GET /users/:uid and set isVIP from user doc
    ApiService.getUserProfile(currentUser.uid).then(u => setIsVIP(u.isVIP || false));
  }, []);
  return <VIPContext.Provider value={{ isVIP }}>{children}</VIPContext.Provider>;
}

export const useVIP = () => useContext(VIPContext);
```

5. Wrap the app navigator in `App.tsx` with `<VIPProvider>`.

6. In `FeedScreen` (or wherever ads are injected into the feed), conditionally skip ad rendering if `useVIP().isVIP === true`.

CONSTRAINTS

- Do not modify any files not listed above
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `GET /venues/:venueId/crowd-history` returns 403 for non-VIP users.
 2. `useVIP().isVIP` is accessible from any screen component.
-

Task 73 — Complete navigation architecture

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 5, Task 73.

PROBLEM

The current navigation is a basic Stack + Bottom Tab setup. All screens built in Phase 3 and 4 must be wired into a complete navigation tree.

FILES TO EDIT

- `PeepIMobile/App.tsx`

EXACT CHANGES REQUIRED

Rebuild the navigation structure in `App.tsx` using `@react-navigation/native-stack` and `@react-navigation/bottom-tabs`:

1. **Auth Stack** (shown when user is not logged in):

tsx

```

<Stack.Navigator screenOptions={{ headerShown: false }}>
  <Stack.Screen name="Onboarding" component={OnboardingScreen} />
  <Stack.Screen name="Login" component={LoginScreen} />
  <Stack.Screen name="Register" component={RegisterScreen} />
  <Stack.Screen name="SignUpConfirmed" component={SignUpConfirmedScreen} />
  <Stack.Screen name="LocationPermission" component={PermissionsScreen} />
</Stack.Navigator>

```

2. **Main Tab Bar** (shown when logged in, `headerShown: false` on tab bar):

tsx

```

<Tab.Navigator>
  <Tab.Screen name="Feed" component={FeedStackNavigator} />
  <Tab.Screen name="Deals" component={DealsScreen} />
  <Tab.Screen name="GetPeeps" component={GetPeepsStackNavigator} />
  <Tab.Screen name="Leaderboard" component={LeaderboardScreen} />
  <Tab.Screen name="Profile" component={ProfileStackNavigator} />
</Tab.Navigator>

```

Tab icons: Feed= 🏠, Deals= 🎉, GetPeeps= 👁, Leaderboard= 🏆, Profile= 👤.

3. **Feed Stack Navigator** (nested under Feed tab): Screens: Feed → VenueScreen → PeepDetail → UserProfile → Likers → Report.
4. **GetPeeps Stack Navigator**: Screens: GetPeeps → MapScreen → VenueScreen (shared).
5. **Profile Stack Navigator**: Screens: Profile → AccountInfo → Settings → VIPeeps → MyPeeps → Favorites → Groups → Followers → Following → Pioneers → Notifications → Search → Share → Invite → Menu.
6. **Modal Stack** (presented as modals over the main tab bar):
 - CreatePeep
 - PioneerCongrat
 - DealClaimed
7. **Merchant Stack** (separate nested navigator, accessed from Profile menu):
 - MerchantSignIn → MerchantSetupStep1 → MerchantSetupStep2 → MerchantAccountNumber → MerchantPortal → MerchantActivity → MerchantAccountInfo → HowToAdvertise.
8. Update `RootStackParamList` in `Navigation.ts` to include all screens and their param types.

CONSTRAINTS

- Do not modify any files not listed above
- Do not change any screen's internal logic — only the navigation wiring
- After changes, the app must compile and all screens must be navigable

VERIFICATION

1. The bottom tab bar shows all 5 tabs and navigates correctly between them.
 2. `CreatePeep` is accessible as a modal from the Feed screen and can be dismissed.
-

Task 74 — Staging environment

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 5, Task 74.

PROBLEM

There is no staging environment. All development is happening against the production Firebase project, which is a serious risk.

FILES TO EDIT

- `.github/workflows/deploy.yml`
- `env.example`
- `PeepIMobile/env.example`

EXACT CHANGES REQUIRED

1. In `.github/workflows/deploy.yml`, update the deployment logic:

```
yaml
```

```

# Deploy to staging on push to main branch
- name: Deploy to Staging
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  run: |
    echo "Deploying to staging..."
    # Add staging deploy commands here (e.g., firebase deploy --project peep1-sta

# Deploy to production only on tagged releases
- name: Deploy to Production
  if: startsWith(github.ref, 'refs/tags/v')
  run: |
    echo "Deploying to production..."
    # Add production deploy commands here (e.g., firebase deploy --project peep1-

```

2. Update `env.example` with all required variables, clearly labeled for staging vs production:

```

# Firebase (use staging values locally)
FIREBASE_PROJECT_ID=peep1-staging
FIREBASE_PRIVATE_KEY=your_private_key_here
FIREBASE_CLIENT_EMAIL=your_client_email_here

# Stripe
STRIPE_SECRET_KEY=sk_test_your_key_here
STRIPE_WEBHOOK_SECRET=whsec_your_secret_here
STRIPE_VIPEEPS_PRICE_ID=price_your_id_here

# App
PORT=3000
NODE_ENV=development

```

3. Update `Peep1Mobile/env.example`:

```

API_BASE_URL=http://localhost:3000
FIREBASE_API_KEY=your_api_key_here
FIREBASE_AUTH_DOMAIN=peep1-staging.firebaseio.com
FIREBASE_PROJECT_ID=peep1-staging
STRIPE_PUBLISHABLE_KEY=pk_test_your_key_here

```

4. Add a comment at the top of `deploy.yml`: `# Staging: deploys on main push. Production: deploys on version tags (e.g. v1.0.0)`.

CONSTRAINTS

- Do not modify any files not listed above
- Do not add actual credentials — only placeholders in example files
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `deploy.yml` deploys to staging on `main` push and to production on version tag.
 2. `env.example` contains all required environment variable names with placeholder values.
-

Task 75 — Error monitoring — Sentry

CONTEXT

PeepI is a real-time crowd intelligence mobile app. This is Phase 5, Task 75.

PROBLEM

There is no error monitoring. Bugs in production are invisible. Sentry must be integrated on both the mobile app and backend.

FILES TO EDIT

- `server.js`
- `PeepIMobile/App.tsx`

EXACT CHANGES REQUIRED

1. Add `@sentry/node` to backend `package.json`. Initialize in `server.js` at the very top (before other imports):

```
js
```

```
const Sentry = require('@sentry/node');
Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV || 'development',
  tracesSampleRate: 1.0,
});
```

2. In `server.js`, find every `catch` block that calls `logger.error()` and add:

```
js

Sentry.captureException(e);
```

immediately before the `logger.error()` call. Do not replace `logger.error()` — add `Sentry.captureException()` alongside it. 3. Add `SENTRY_DSN` to `env.example` as a placeholder. 4. Add `@sentry/react-native` to `PeepMobile/package.json`. Initialize in `App.tsx` before the root component renders:

```
ts

import * as Sentry from '@sentry/react-native';
Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: __DEV__ ? 'development' : 'production',
});
```

5. Wrap the root component export with `Sentry.wrap()`:

```
ts

export default Sentry.wrap(App);
```

CONSTRAINTS

- Do not modify any files not listed above
- Do not replace existing error logging — add Sentry alongside it
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `server.js` initializes Sentry at the top and calls `Sentry.captureException()` in every catch block.
 2. `App.tsx` initializes Sentry and wraps the root component.
-

Task 76 — Analytics — Mixpanel

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 5, Task 76.

PROBLEM

There is no analytics. User behavior is invisible. Mixpanel must be integrated on the mobile app to track key events.

FILES TO EDIT

- `PeeplMobile/App.tsx`
- All key screens (add analytics calls where the relevant events occur)

EXACT CHANGES REQUIRED

1. Add `mixpanel-react-native` to `PeeplMobile/package.json`. Initialize in `App.tsx`:

ts

```
import { Mixpanel } from 'mixpanel-react-native';
const mixpanel = new Mixpanel(process.env.MIXPANEL_TOKEN, true);
mixpanel.init();
```

2. Export the `mixpanel` instance from `App.tsx` or create `PeeplMobile/src/services/AnalyticsService.ts`:

ts

```
import { Mixpanel } from 'mixpanel-react-native';
const analytics = new Mixpanel(process.env.MIXPANEL_TOKEN || '', true);
analytics.init();
export default analytics;
```

3. Add the following tracking calls in the specified locations:

- `App.tsx` on app launch: `analytics.track('app_opened')`.
- `RegisterScreen.tsx` on successful registration:
`analytics.track('sign_up_completed', { method: 'email' })`.
- `CreatePeepScreen.tsx` on successful submission:
`analytics.track('peep_created', { crowdSize, vibe, hasPioneer: isPioneer, hasPhoto: !!photoUrl })`.
- `PioneerCongratScreen.tsx` on mount: `analytics.track('pioneer_earned', { venueName })`.
- `DealClaimedScreen.tsx` on mount: `analytics.track('deal_claimed', { merchantName, adId })`.
- `MerchantPortalScreen.tsx` on successful ad submission:
`analytics.track('merchant_ad_submitted', { rateType, totalCost })`.
- `VIpeepsScreen.tsx` on subscribe button tap:
`analytics.track('vipeeps_started')`.
- `SearchScreen.tsx` on search submit: `analytics.track('search_performed', { query: term })`.

4. Add `MIXPANEL_TOKEN` to `Peep1Mobile/env.example`.

CONSTRAINTS

- Do not modify any files not listed above
- Do not add tracking to every component — only the 8 events listed
- Maintain existing code style and conventions
- After changes, the app must still compile

VERIFICATION

1. `AnalyticsService.ts` exists and exports the initialized Mixpanel instance.
2. `peep_created` is tracked with `crowdSize` and `hasPioneer` properties in `CreatePeepScreen`.

Task 77 — App Store assets and metadata

CONTEXT

Peep1 is a real-time crowd intelligence mobile app. This is Phase 5, Task 77.

PROBLEM

No App Store or Google Play assets exist. These must be created before submission.

FILES TO EDIT

- App store listings (external tools — Figma, App Store Connect, Google Play Console)

EXACT CHANGES REQUIRED

This task is executed outside of Cursor. Complete the following:

1. App icon:

- Design: Blue (`#1565C0`) background, white `"peep1"` wordmark in bold lowercase.
- Export at all required sizes for iOS (1024×1024 for App Store Connect, plus all `AppIcon.appiconset` sizes in `ios/Peep1Mobile/Images.xcassets/AppIcon.appiconset/`).
- Export at all required sizes for Android (`mipmap-hdpi` through `mipmap-xxxhdpi` in `android/app/src/main/res/`).

2. Splash screen:

- Blue background with centered white `"peep1"` wordmark.
- Configure using `react-native-splash-screen` or the React Native 0.71+ built-in.

3. App Store description (for App Store Connect):

Know before you go.

Peep1 is the real-time crowd intelligence app. See how busy any venue is, right now, updated by real people.

- See live crowd levels at bars, restaurants, parks, and events
- Post a Peep in seconds from anywhere
- Earn Pioneer status as the first to report a venue
- Discover deals and offers from local merchants
- Compete on the leaderboard with friends

Keywords: crowd, busy, venue, nightlife, bar, restaurant, events, real-time, people, pioneer

- ### 4. Screenshots:
- Use the rendered screens from the app (at minimum: feed screen, venue screen, create peep screen, leaderboard, deals screen). Export at iPhone 6.7" size

(1290×2796).

5. **Google Play Store listing:** Same description and screenshots adapted for Play Console format.

6. **Configure signing:**

- iOS: Set up Distribution certificate and provisioning profile in Xcode.
- Android: Create release keystore: `keytool -genkey -v -keystore peep1-release.keystore -alias peep1 -keyalg RSA -keysize 2048 -validity 10000`.
Store keystore and password in a secure location (not in the repo).

CONSTRAINTS

- Do not commit the Android keystore file to the repository
- Do not commit signing credentials — use environment variables or GitHub secrets

VERIFICATION

1. The app icon displays correctly on an iOS simulator home screen.
 2. The splash screen appears on app launch and dismisses correctly.
-

Task 78 — Apple and Google developer accounts

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 5, Task 78. **START THIS TASK DURING PHASE 3 — processing takes time.**

PROBLEM

Developer accounts are required before app store submission and take time to process. Both must be set up as early as possible.

FILES TO EDIT

- No code files. This is an external account setup task.

EXACT CHANGES REQUIRED

1. **Apple Developer Program:**

- Enroll at `developer.apple.com/enroll`. Cost: \$99/year.
- Choose individual or organization enrollment based on your business structure.

- Complete identity verification (can take 24–48 hours).
- Once approved, create an App ID for `com.peepl.app` in Certificates, Identifiers & Profiles.
- Create a Distribution Certificate.
- Create an App Store Distribution Provisioning Profile.
- Create the app record in App Store Connect.

2. Google Play Console:

- Register at `play.google.com/console`. Cost: \$25 one-time fee.
- Complete developer account verification.
- Create a new app record for Peepl.
- Complete the store listing (can be partial at first — complete in Task 77).
- Configure content rating questionnaire.

3. Document in the project README (add a section `## App Store Accounts`):

- Apple Team ID: [fill in after enrollment]
- Apple Bundle ID: `com.peepl.app`
- Google Play Package Name: `com.peepl.app`

CONSTRAINTS

- Do not commit any credentials, certificates, or keystores to the repository
- Store all signing materials in a secure password manager or GitHub Secrets

VERIFICATION

1. Apple Developer Program enrollment is approved and an App ID exists for `com.peepl.app`.
2. Google Play Console app record is created and the package name `com.peepl.app` is registered.

Task 79 — Privacy Policy and Terms of Service

CONTEXT

Peepl is a real-time crowd intelligence mobile app. This is Phase 5, Task 79. **START THIS TASK DURING PHASE 3 — required before app store submission.**

PROBLEM

Legal documents are required by both Apple and Google before an app can be submitted. These documents must cover all of Peepl's specific data practices.

FILES TO EDIT

- No code files. This is a legal document creation task.

EXACT CHANGES REQUIRED

Engage a lawyer or use a reputable generator (e.g., Termly, iubenda) to create documents that must cover at minimum:

1. Privacy Policy must address:

- Location data: collected in real-time for venue detection and proximity features. Stored as coordinates on Firestore. Not sold to third parties.
- User-generated content (UGC): photos and text posted by users. Who owns them, how Peepl can use them.
- Geofencing notifications: how geofencing data is collected and when push notifications are triggered.
- Merchant advertising disclosure: that merchants pay to display ads and how those ads are identified.
- GDPR compliance: right to access, right to erasure, data portability. Include the `"Delete my data"` endpoint documentation.
- CCPA compliance: California users' right to know and delete.
- Firebase/Google data processing: disclose use of Firebase Auth, Firestore, and Storage.
- Mixpanel/Sentry disclosure: analytics and error monitoring data collected.

2. Terms of Service must address:

- User conduct: no false crowd reports, no spam.
- UGC ownership and license grant to Peepl.
- Merchant advertising terms: rate structure, no refund policy for time elapsed.
- VIPeeps subscription: billing terms, cancellation policy.
- Account termination rights.
- Limitation of liability.

3. Host both documents at publicly accessible URLs:

- `https://peepl.app/privacy` (or use a link shortener until the domain is set up)

- `https://peopl.app/terms`

4. **Link both documents** in `RegisterScreen.tsx` (already referenced in Task 27) and in `SettingsScreen.tsx`.

CONSTRAINTS

- Consult an actual lawyer — do not rely solely on templates for an app that handles location data and payments
- Both URLs must be live and publicly accessible before App Store submission

VERIFICATION

1. Both documents are accessible at public URLs.
 2. `RegisterScreen.tsx` and `SettingsScreen.tsx` link to the live document URLs.
-

Task 80 — Final end-to-end QA pass

CONTEXT

Peopl is a real-time crowd intelligence mobile app. This is Phase 5, Task 80 — the final gate before app store submission.

PROBLEM

All features are built but have not been tested as a complete integrated system on real devices. Every critical user flow must be manually tested and all bugs fixed before submission.

FILES TO EDIT

- Any files containing bugs discovered during this QA pass.

EXACT CHANGES REQUIRED

Test the following complete flows on BOTH iOS simulator AND Android simulator AND at least one physical device of each platform:

1. Sign-up flow:

- Open app → Onboarding (3 screens) → Register → `SignUpConfirmed` → Location permission → Push permission → Feed. Verify each screen transitions correctly.

2. First Peep + Pioneer flow:

- Tap Create Peep → fill all fields (crowdSize, M/F ratio, A/K ratio, age ranges, vibe, crowdTrend, notes, photo) → Submit.
- If first peep at venue → `PioneerCongrat` screen appears with correct venue name and +50 points.
- Verify the peep appears in the feed.
- Verify `points` on the user's document increased correctly.

3. Leaderboard update:

- After creating a peep, navigate to Leaderboard → Verify the current user appears with correct points.

4. Merchant sign-up flow:

- Navigate to Merchant Sign In → Set Up Account → Step 1 (fill business info, select category) → Step 2 (fill contact info) → Submit.
- `MerchantAccountNumber` screen shows a 7-digit number.
- Navigate to Merchant Portal → Create an ad (fill offer, set times, select rate, review, submit).
- Wait for ad status to flip to `"live"` (or manually update in Firestore for testing).

5. Ad in consumer feed:

- On the consumer feed, verify the live merchant ad appears after every 2nd peep.
- Tap `"Claim"` → `DealClaimed` screen shows with countdown.

6. VIPeeps subscription:

- Navigate to VIPeeps → Tap subscribe → Complete Stripe test payment → Verify `isVIP: true` on user doc → Confirm ads are hidden from feed.

7. For every bug found: Fix it in the appropriate file. Re-run the relevant flow. Do not submit until all flows pass cleanly on both platforms.

CONSTRAINTS

- Fix bugs in the appropriate files — do not work around bugs with hacks
- Do not add any new features in this task
- After all flows pass: `git tag v1.0.0 && git push --tags`

VERIFICATION

1. All 5 flows above complete without crashes or errors on iOS and Android.

2. The repo is tagged `v1.0.0` and the GitHub Actions `deploy.yml` triggers a production deployment.
-

End of all 80 Cursor Composer prompts for the Peepl build plan.

Commit format reminder: `[Phase X / Task Y] Description`

Ground rule: One task at a time. Complete it, test it, commit it.